

Redundancy and equality test in the PMNS, application to Elliptic Curve Diffie–Hellman

Fangan Yssouf Dosso¹, Alexandre Berzati², Nadia El Mrabet³ and Julien Proy²

¹ SealSQ, Meyreuil, France, ydosso@sealsq.com

² Thales DIS, Meyreuil, France, {alexandre.berzati,julien.proy}@thalesgroup.com

³ SAS laboratory, École des Mines de Saint-Étienne, Gardanne, France,
nadia.el-mrabet@emse.fr

Abstract. The Polynomial Modular Number System (PMNS) is a non-positional number system for modular arithmetic. A PMNS is defined by a tuple (p, n, γ, ρ, E) , where p , n , γ and ρ are positive non-zero integers and $E \in \mathbb{Z}_n[X]$ is a monic polynomial such that $E(\gamma) \equiv 0 \pmod{p}$. The PMNS is a redundant number system. In [DDE⁺], Didier et al. used this redundancy property to randomise the data during the Elliptic Curve Scalar Multiplication (ECSM). In this paper, we refine the results on redundancy and propose several new results on PMNS. More precisely, we study a generalisation of the Montgomery-like internal reduction method proposed in [NP08], along with some improvements on parameter bounds for smaller memory cost to represent elements in this system. We also show how to perform equality test in the PMNS. This open problem has been a limitation for the interesting use of PMNS in cryptography. Finally, as an application, we show the use of the Elliptic Curve Diffie Hellman key agreement protocol with PMNS, without needing another number system. We also provide SageMath and C implementations. We make some comparisons with the OpenSSL library.

Keywords: Modular arithmetic, Polynomial Modular Number System · Internal reduction · Euclidean lattices · Redundancy · Equality test · Elliptic Curve Diffie-Hellman

1 Introduction

Modular arithmetic is one of the key points for efficient and secure cryptographic applications, such as Elliptic Curve Cryptography (ECC) [Kob87, Mil85], RSA [RSA78] and some post-quantum schemes such as Crystal-Dilithium [DKL⁺18]. Many approaches have been proposed to perform it both efficiently and safely. When the modulus can be chosen freely, special numbers such as Mersenne and pseudo-Mersenne numbers [Cra92, Sol99] can be used. They are very efficient for modular reduction. In many applications, such as pairing-based cryptography [EJ17], the modulus cannot be chosen arbitrarily and does not have a shape that would allow speeding up modular arithmetic. For such moduli, there are general modular reduction methods, such as those proposed by Barrett [Bar] and Montgomery [Mon85]. These methods are based on the classical binary number system. To further improve this modular arithmetic, alternative number systems have been proposed. A famous one is the Residue Number System (RNS) [Gar59], which has been widely studied and implemented. It is an efficient number system which has a very high parallelisation capability. In 2004, Bajard, Imbert and Plantard proposed the Polynomial Modular Number System (PMNS) [BIPb, BIPa] for modular arithmetic modulo a prime

integer p . In this system, elements are polynomials. More precisely, we have the following definition.

Definition 1. A polynomial modular number system (PMNS) is defined by a tuple $\mathcal{B} = (p, n, \gamma, \rho, E)$, such that E is a monic polynomial in $\mathbb{Z}[X]$ of degree n , having γ as a root modulo p , i.e. $E(\gamma) \equiv 0 \pmod{p}$, and for each integer $y \in \mathbb{Z}/p\mathbb{Z}$ there exists a polynomial $V(X) = v_0 + v_1X + \dots + v_{n-1}X^{n-1}$ such that:

$$y = V(\gamma) \pmod{p},$$

where $|v_i| < \rho$ and $2 \leq \rho, \gamma < p$. In this case, we say that $V(X)$ (or equivalently the vector $V = (v_0, \dots, v_{n-1})$) is a representation of y in $\mathcal{B}$.

In the PMNS, polynomial multiplications are performed modulo E . The reduction modulo E is called **external reduction** and E is called *the external reduction polynomial*.

After an addition or multiplication (modulo E), the result might not be in the PMNS, since its coefficients are likely to be greater than ρ , especially after a multiplication. To keep the coefficients of the result bounded by ρ , an operation called **internal reduction** is performed. It is somehow equivalent to the modular reduction in classical binary representation. In Section 2.5, we describe in detail the Montgomery-like internal reduction method proposed by Negre and Plantard [NP08].

Like the RNS, the PMNS offers a very high parallelisation capability, due to the polynomial structure of its elements. In [DDV20, M  l22, DRV22], it is shown that the PMNS can be an efficient alternative to the classical representation, for integer sizes used in ECC. It is also shown that operations in PMNS can be vectorized very efficiently. In [DRDE], sequential and vectorized software implementations of PMNS and RNS are compared, for small to large moduli. In [CDD⁺21, NEPV24], efficient hardware implementations of PMNS are presented.

The PMNS is a redundant number system. In fact, an element $a \in \mathbb{Z}/p\mathbb{Z}$ can have more than one representation. For example, in the PMNS $\mathcal{B} = (19, 3, 7, 2, X^3 - 1)$, the polynomials $(-X - 1)$ and X^2 are representations of 11. In [DDE⁺], Didier et al. use this redundancy property to randomize arithmetic and conversion operations in order to randomize the data during the Elliptic Curve Scalar Multiplication (ECSM), which could be very interesting to secure implementations against some Side Channel Attacks. However, this redundancy property of PMNS has a drawback. It makes the equality check non-trivial. Indeed, since elements can have more than one representation, a simple comparison of two polynomials is not sufficient to conclude that they represent two different elements in $\mathbb{Z}/p\mathbb{Z}$. Until now, the best way to determine if two polynomials $A, B \in \mathbb{Z}_{n-1}[X]$ represent the same element $a \in \mathbb{Z}/p\mathbb{Z}$ is to first compute $C = A - B$, then evaluate C modulo p to check if $C(\gamma) \equiv 0 \pmod{p}$. So, a conversion out of the PMNS (the evaluation of C) is required, which is not appropriate for some cryptographic use cases where equality testing is, for instance, done during the process, to detect fault attacks [BOS, Joy19] or for signature verification [FS07].

Contributions This paper is a comprehensive study of how elements are represented in the PMNS and how they behave with respect to the arithmetic and conversion operations. We aim to provide a complete understanding of this system. To do so, we introduce a generalised Montgomery-like internal reduction method with several properties. This is the first major (and rather theoretical) contribution of this paper. It is a toolkit of results that will serve as the basis for our following three major (and practical) contributions:

- Improved bounds to represent PMNS elements for a smaller memory cost.
- Precise study of redundancy in PMNS, explaining the relation between representations.

- Equality test within the PMNS.

These contributions make the PMNS a more complete system and allows to compare PMNS elements and remove the burden of conversion to the original system.

Finally, as an application, we show the use of the Elliptic Curve Diffie-Hellman key agreement protocol with PMNS, without needing another number system. This is validated by a SageMath implementation. We also present an efficient C implementation and compare it with the OpenSSL library.

Paper organisation This paper is structured as follows. Section 2 gives some background on PMNS. Section 3 introduces **GMont-like**, a generalised Montgomery-like internal reduction method with a number of properties. In Section 4, we present a set of properties for coefficient reductions in some special domains, which are interesting for both memory requirement and redundancy in the PMNS. These two theoretical sections constitute the toolkit for the next three sections, which respectively focus on memory improvement, redundancy and equality test in the PMNS. Section 8 addresses the related computational operation costs. Before concluding, Section 9 presents the use of the Elliptic Curve Diffie-Hellman key agreement protocol with PMNS. This section also discusses implementations and comparisons.

2 Background on PMNS

For consistency, we assume that $p \geq 3$ and $n \geq 2$. Let us start with the notations we will be using throughout this paper.

2.1 Some notations and conventions

- Let $\mathcal{A}$ be a set, $\#\mathcal{A}$ denotes the number of elements of $\mathcal{A}$.
- $\mathbb{Z}_{n-1}[X]$ is the set of polynomials in $\mathbb{Z}[X]$ with degrees lower than or equal to $n-1$:

$$\mathbb{Z}_{n-1}[X] = \{C \in \mathbb{Z}[X] \mid \deg(C) \leq n-1\}.$$

- If $A \in \mathbb{Z}_{n-1}[X]$, we assume that $A(X) = a_0 + a_1X + \dots + a_{n-1}X^{n-1}$ can equivalently be represented as the vector $A = (a_0, \dots, a_{n-1}) \in \mathbb{Z}^n$.
- Let $a \in \mathbb{Z}/p\mathbb{Z}$. If a polynomial $A \in \mathbb{Z}_{n-1}[X]$ is such that $A(\gamma) \equiv a \pmod{p}$, then we say that A is a representation of a in $\mathcal{B}$.
- If $\alpha = (\alpha_0, \dots, \alpha_{n-1})$, then $\alpha \pmod{\phi} = (\alpha_0 \pmod{\phi}, \dots, \alpha_{n-1} \pmod{\phi})$.
- Let $C \in \mathbb{Z}[X]$. Then $C \bmod (E, \phi)$ is the polynomial reduction $C \bmod E$, with the coefficients of the result are computed modulo ϕ .
- Unless otherwise specified, the symbol $\mathcal{B}$ will always designate a PMNS (p, n, γ, ρ, E) .
- Let $A \in \mathbb{Z}^n$ be a vector. We have:

$$\|A\|_\infty = \max_{0 \leq i \leq n-1} |a_i|.$$
- Let $\mathcal{W} \in \mathbb{Z}^{n \times n}$ be a matrix. We have:

$$\|\mathcal{W}\|_1 = \max_{0 \leq j \leq n-1} \sum_{i=0}^{n-1} |w_{ij}|.$$

2.2 PMNS and Euclidean lattices

Given a tuple $\mathcal{B} = (p, n, \gamma, \rho, E)$, one can build the n -dimensional full-rank Euclidean lattice $\mathcal{L}_{\mathcal{B}}$ defined as follows:

$$\mathcal{L}_{\mathcal{B}} = \{A \in \mathbb{Z}_{n-1}[X] \mid A(\gamma) \equiv 0 \pmod{p}\}. \quad (1)$$

It is the set of polynomials with degrees strictly less than n and having γ as a root modulo p . A basis of $\mathcal{L}_{\mathcal{B}}$ is the $n \times n$ matrix $\mathbf{B}$, defined as follows:

$$\mathbf{B} = \begin{pmatrix} p & 0 & 0 & \dots & 0 & 0 \\ t_1 & 1 & 0 & \dots & 0 & 0 \\ t_2 & 0 & 1 & \dots & 0 & 0 \\ \vdots & & & \ddots & & \vdots \\ t_{n-2} & 0 & 0 & \dots & 1 & 0 \\ t_{n-1} & 0 & 0 & \dots & 0 & 1 \end{pmatrix} \begin{matrix} \leftarrow p \\ \leftarrow X + t_1 \\ \leftarrow X^2 + t_2 \\ \\ \leftarrow X^{n-2} + t_{n-2} \\ \leftarrow X^{n-1} + t_{n-1} \end{matrix}, \quad (2)$$

where $t_i = (-\gamma^i) \bmod p$.

The following result, from [BMPV22], gives a condition on ρ for $\mathcal{B}$ to be a PMNS.

Theorem 1. *Let $p \geq 3$, $n \geq 1$ be two integers, $\gamma \in \mathbb{Z}/p\mathbb{Z} \setminus \{\bar{0}\}$ and $E \in \mathbb{Z}_n[X]$ a monic polynomial such that $E(\gamma) \equiv 0 \pmod{p}$. Let $\mathcal{W}$ be any basis of the lattice $\mathcal{L}_{\mathcal{B}}$. A tuple $\mathcal{B} = (p, n, \gamma, \rho, E)$ defines a PMNS if:*

$$\rho > \frac{1}{2} \|\mathcal{W}\|_1.$$

Remark 1. Theorem 1 works with any basis of $\mathcal{L}_{\mathcal{B}}$ regardless of its quality, defined here as its 1-norm. In practice however, we want ρ to be as small as possible for small memory cost. So, one should take $\mathcal{W}$ as a reduced basis of $\mathcal{L}_{\mathcal{B}}$ (the smaller the better). Such a basis can be obtained with algorithms like LLL [LLL82], BKZ[SE91] or HKZ[KI77], applied to the basis $\mathbf{B}$ (Equation 2). This results in a parameter ρ such that $\rho \approx \sqrt[n]{p}$.

2.3 A property on $\mathcal{L}_{\mathcal{B}}$ sublattices

Let $\mathcal{L}$ be a sublattice of $\mathcal{L}_{\mathcal{B}}$, having $\mathcal{G}$ as a basis. Assuming that $\mathcal{G}_i$ is the i^{th} row of $\mathcal{G}$, let us consider the fundamental regions $\mathcal{H}$ and $\mathcal{H}'$ defined as follows:

$$\mathcal{H} = \{t \in \mathbb{R}^n \mid t = \sum_{i=0}^{n-1} \mu_i \mathcal{G}_i \text{ and } 0 \leq \mu_i < 1\}, \quad (3)$$

$$\mathcal{H}' = \{t \in \mathbb{R}^n \mid t = \sum_{i=0}^{n-1} \mu_i \mathcal{G}_i \text{ and } -\frac{1}{2} \leq \mu_i < \frac{1}{2}\}. \quad (4)$$

Lemma 1. *Let $a \in \mathbb{Z}/p\mathbb{Z}$. There exists $A \in \mathcal{H} \cap \mathbb{Z}^n$ (resp. $A \in \mathcal{H}' \cap \mathbb{Z}^n$), such that: $a = A(\gamma) \pmod{p}$.*

Proof. Let $V = (a, 0, \dots, 0)$. From lattice theory, we know that there exists a vector $T \in \mathcal{L}$ such that $V + T \in \mathcal{H}$ (resp. $V + T \in \mathcal{H}'$). Let $A = V + T$. Since $\mathcal{L} \subseteq \mathcal{L}_{\mathcal{B}} \subset \mathbb{Z}^n$, $T \in \mathbb{Z}^n$. So, $A \in \mathbb{Z}^n$. Additionally, $T(\gamma) \equiv 0 \pmod{p}$, since $T \in \mathcal{L}$.

Thus, $A(\gamma) \equiv V(\gamma) \pmod{p}$. $\square$

This lemma says that every element $a \in \mathbb{Z}/p\mathbb{Z}$ has at least one representation in $\mathcal{H} \cap \mathbb{Z}^n$ (resp. $\mathcal{H}' \cap \mathbb{Z}^n$).

Remark 2. Let $A \in \mathbb{Z}^n$.

- If $A \in \mathcal{H}$, then $\|A\|_{\infty} < \|\mathcal{G}\|_1$.
- If $A \in \mathcal{H}'$, then $\|A\|_{\infty} \leq \frac{1}{2} \|\mathcal{G}\|_1$.

2.4 External reduction

Let $\mathcal{B} = (p, n, \gamma, \rho, E)$ be a PMNS, where $E \in \mathbb{Z}_n[X]$ is a monic polynomial. As explained in the introduction, polynomial multiplications in PMNS are done modulo E . The reduction modulo E is called **external reduction**.

When $E(X) = X^n - \lambda$, with $\lambda \in \mathbb{Z} \setminus \{0\}$ and $|\lambda|$ small, this operation can be done very efficiently, as explained in [BIPb].

Let us assume that $E(X) = X^n + e_{n-1}X^{n-1} + \dots + e_1X + e_0$. To optimise the general case for E , the authors in [DRV22] introduce the **external reduction matrix** $\mathcal{E}$, which is defined as follows:

$$\mathcal{E} = \begin{pmatrix} -e_0 & -e_1 & \dots & -e_{n-1} \\ \dots & \dots & \dots & \dots \\ \vdots & \vdots & & \vdots \\ \dots & \dots & \dots & \dots \end{pmatrix} \begin{matrix} \leftarrow X^n \bmod E \\ \leftarrow X^{n+1} \bmod E \\ \leftarrow X^{2n-2} \bmod E \end{matrix}. \quad (5)$$

It is a $(n-1) \times n$ matrix where each row contains the coefficients of the polynomial $X^{n+i} \pmod{E}$, for $i = 0, \dots, n-2$.

Let $A, B \in \mathbb{Z}_{n-1}[X]$ be two polynomials. Let $C = A \times B = c_0 + c_1X + \dots + c_{2n-2}X^{2n-2}$ and $R = C \bmod E$. In [DRV22], it is shown that:

$$R = (c_0, \dots, c_{n-1}) + (c_n, \dots, c_{2n-2})\mathcal{E}. \quad (6)$$

Moreover, with $\mathcal{E}'$ being the $(n-1) \times n$ matrix such that $\mathcal{E}'_{ij} = |\mathcal{E}_{ij}|$, it is also shown that:

$$\|R\|_\infty \leq w \|A\|_\infty \|B\|_\infty, \quad (7)$$

with:

$$w = \|(1, 2, \dots, n) + (n-1, n-2, \dots, 1)\mathcal{E}'\|_\infty. \quad (8)$$

In [DRV22] (Table 1), a set of polynomials E minimising w , and building very sparse and multiplication-friendly matrices $\mathcal{E}$ are given.

2.5 Internal reduction

Many approaches have been proposed to perform the internal reduction [BIPb, BIPa, NP08, M  l22]. In this paper, we focus on the Montgomery-like method proposed in [NP08]. It introduces three parameters: an integer $\phi \geq 2$ and two polynomials $M, M' \in \mathbb{Z}_{n-1}[X]$ such that: $M(\gamma) \equiv 0 \pmod{p}$ and $M' = -M^{-1} \bmod (E, \phi)$. Algorithm 1 describes this method. Notice that this method introduces a factor ϕ^{-1} on the output. So, a conversion in Montgomery domain should be done, as explained in [DDV20](Section 4.4).

Algorithm 1 Coefficients reduction [NP08]

Require: $V \in \mathbb{Z}_{n-1}[X]$, $\mathcal{B} = (p, n, \gamma, \rho, E)$, $M \in \mathbb{Z}_{n-1}[X]$ such that $M(\gamma) \equiv 0 \pmod{p}$, $\phi \in \mathbb{N} \setminus \{0, 1\}$ and $M' = -M^{-1} \bmod (E, \phi)$.

Ensure: $S(\gamma) = V(\gamma)\phi^{-1} \pmod{p}$, with $S \in \mathbb{Z}_{n-1}[X]$

- 1: $Q \leftarrow V \times M' \bmod (E, \phi)$
 - 2: $T \leftarrow Q \times M \bmod E$
 - 3: $S \leftarrow (V + T)/\phi$
 - 4: return S
-

In [DDV20], focusing on the case $E(X) = X^n - \lambda$, the authors show that this approach is at least as efficient as the classical Montgomery modular reduction method. In [DRV22], the authors extend that work to any monic polynomial $E \in \mathbb{Z}_n[X]$. To do that efficiently,

regardless of E 's shape, they introduce the matrices $\mathcal{M}$ and $\mathcal{M}'$ defined below. With these matrices, Algorithm 1 is rewritten as Algorithm 2.

$$\mathcal{M} = \begin{pmatrix} m_0 & m_1 & \dots & m_{n-1} \\ \dots & \dots & \dots & \dots \\ \vdots & \vdots & & \vdots \\ \dots & \dots & \dots & \dots \end{pmatrix} \begin{array}{l} \leftarrow M \\ \leftarrow X \times M \bmod E \\ \leftarrow X^{n-1} \times M \bmod E \end{array}, \quad (9)$$

$$\mathcal{M}' = \begin{pmatrix} m'_0 & m'_1 & \dots & m'_{n-1} \\ \dots & \dots & \dots & \dots \\ \vdots & \vdots & & \vdots \\ \dots & \dots & \dots & \dots \end{pmatrix} \begin{array}{l} \leftarrow M' \\ \leftarrow X \times M' \bmod (E, \phi) \\ \leftarrow X^{n-1} \times M' \bmod (E, \phi) \end{array}.$$

Algorithm 2 Coefficients reduction for PMNS (**RedCoeff**)

Require: $V \in \mathbb{Z}_{n-1}[X]$, the matrices $\mathcal{M}$, $\mathcal{M}'$ and $\phi \in \mathbb{N} \setminus \{0, 1\}$.

Ensure: $S(\gamma) = V(\gamma)\phi^{-1} \pmod{p}$, with $S \in \mathbb{Z}_{n-1}[X]$

- 1: $Q = (v_0, \dots, v_{n-1})\mathcal{M}' \pmod{\phi}$
 - 2: $T = (q_0, \dots, q_{n-1})\mathcal{M}$
 - 3: $S \leftarrow (V + T)/\phi$
 - 4: return S
-

2.6 Arithmetic and conversion operations

In this section, we briefly recall the main arithmetic and conversion operations in the PMNS.

2.6.1 Addition and subtraction

Let $A, B \in \mathcal{B}$ be two polynomials. The addition (resp. subtraction) is a simple polynomial addition (resp. subtraction); i.e. the operation $C = A + B$ (resp. $C = A - B$). Although $\deg(C) < n$, C might not be in $\mathcal{B}$, since we only have $\|C\|_\infty < 2\rho$. So, an internal reduction should be done on C to guarantee that the result is in the PMNS. However, this reduction (see Algorithm 2) is too expensive compared to the polynomial addition or subtraction.

To avoid this need for coefficient reductions, a parameter δ has been introduced in [DDV20]. It is the maximum number of consecutive additions of elements in $\mathcal{B}$ that we want to compute before doing a modular multiplication.

If C is the result of such successive additions, then $\|C\|_\infty < (\delta + 1)\rho$. In Section 2.6.4, we recall the bounds proposed in [DRV22] to ensure complete coefficient reductions of the product of such polynomials.

2.6.2 Modular multiplication

Multiplication is a simple polynomial multiplication, followed by the external reduction and then the internal reduction, see Algorithm 3.

2.6.3 Conversion operations

The use of PMNS requires conversions from and to the classical binary representation. These operations are described in detail in [BIPb](Section 4.4) and [DRV22](Section 6.1).

Algorithm 3 Modular multiplication for PMNS**Require:** $A, B \in \mathbb{Z}_{n-1}[X]$, $\mathcal{B} = (p, n, \gamma, \rho, E)$, the matrices $\mathcal{M}$, $\mathcal{M}'$, $\mathcal{E}$ and $\phi \in \mathbb{N} \setminus \{0, 1\}$.**Ensure:** $S(\gamma) = A \times B(\gamma)\phi^{-1} \pmod{p}$, with $S \in \mathbb{Z}_{n-1}[X]$

- 1: $C = A \times B$
- 2: $V = (c_0, \dots, c_{n-1}) + (c_n, \dots, c_{2n-2})\mathcal{E}$
- 3: $S \leftarrow \text{RedCoeff}(V)$
- 4: return S

The conversion from PMNS is a polynomial evaluation modulo p . Here, we briefly present the conversion process to PMNS.

Algorithms 4 and 5 describe two ways to convert an element from $\mathbb{Z}/p\mathbb{Z}$ to $\mathcal{B}$. Algorithm 5 is fast (slightly faster than a modular multiplication), but requires precomputed polynomials P_i , such that $P_i(\gamma) \equiv (\rho^i \phi^2) \pmod{p}$, for $i = 0 \dots n-1$. Algorithm 4 is relatively slower (it makes n calls to RedCoeff, whereas Algorithm 5 makes only one), but only requires to precompute $\tau = \phi^n \pmod{p}$. In practice, Algorithm 4 is used to precompute the polynomials P_i and then Algorithm 5 is used for fast conversion.

Algorithm 4 Exact conversion from binary representation to PMNS**Require:** $a \in \mathbb{Z}/p\mathbb{Z}$, $\mathcal{B} = (p, n, \gamma, \rho, E)$ and $\tau = \phi^n \pmod{p}$ **Ensure:** $A \in \mathcal{B}$, such that $A \equiv a_{\mathcal{B}}$

- 1: $\alpha = a \times \tau \pmod{p}$
- 2: $A = (\alpha, 0, \dots, 0)$ # a polynomial of degree 0
- 3: **for** $i = 0 \dots n-1$ **do**
- 4: $A \leftarrow \text{RedCoeff}(A)$
- 5: **end for**
- 6: return A

Algorithm 5 Fast conversion from binary representation to PMNS**Require:** $a \in \mathbb{Z}/p\mathbb{Z}$, $\mathcal{B} = (p, n, \gamma, \rho, E)$ and $P_i \in \mathcal{B}$ such that $P_i(\gamma) \equiv (\rho^i \phi^2) \pmod{p}$ **Ensure:** $A \in \mathcal{B}$, such that $A \equiv (a\phi)_{\mathcal{B}}$

- 1: $t = (t_{n-1}, \dots, t_0)_{\rho}$ # radix- ρ decomposition of a
- 2: $U \leftarrow \sum_{i=0}^{n-1} t_i P_i(X)$
- 3: $A \leftarrow \text{RedCoeff}(U)$
- 4: return A

In Algorithm 5 (line 1), a radix- ρ decomposition is done. For this operation to be fast, ρ must be a power of two, which is always possible (see Section 2.6.4).

Remark 3. Note that in Algorithm 4, if we take the input $\tau = \phi^{n+1} \pmod{p}$ (instead of $\phi^n \pmod{p}$), then the output of this algorithm is the same as that of Algorithm 5.

Similarly, in Algorithm 5, if we take the input $P_i(\gamma) \equiv (\rho^i \phi) \pmod{p}$, instead of $(\rho^i \phi^2) \pmod{p}$, then the output will be the same as that of Algorithm 4.

2.6.4 Bounds for consistency

In [DRV22], the authors give these bounds on ρ and ϕ to ensure that the arithmetic and conversion operations in the PMNS are consistent:

$$\rho \geq 2\|\mathcal{M}\|_1 \text{ and } \phi \geq 2w\rho(\delta + 1)^2.$$

Consistent here means that the conversion algorithms to PMNS outputs are in PMNS and also that Algorithm 3 applied to inputs A, B such that $\|A\|_\infty, \|B\|_\infty < (\delta + 1)\rho$, returns a result in the PMNS. In [DRV22], the authors suggest using $\rho = 2^{\lceil \log_2(2\|\mathcal{M}\|_1) \rceil}$, since we want it to be a power of two for fast radix- ρ decomposition.

2.7 PMNS generation

Given a modulus p , one chooses the parameter n . Since $\rho \approx \sqrt[n]{p}$, this gives the approximate size of element coefficients in the PMNS that will be generated. We do not deal with PMNS generation in this paper, this is detailed in [DDV20] for $E(X) = X^n - \lambda$, where $\lambda \in \mathbb{Z} \setminus \{0\}$, and extended in [DRV22] to any monic polynomial $E \in \mathbb{Z}_n[X]$.

In the next two sections, we introduce a method called **GMont-like**, with a set of properties based on it. We will then use these results in Section 5 and subsequent sections, to reduce memory requirements and perform equality test in the PMNS.

3 Montgomery-like internal reduction generalised

The matrix $\mathcal{M}$ (Equation 9) generates the sublattice $\mathcal{L} = \{ZM \bmod E \mid Z \in \mathbb{Z}_{n-1}[X]\}$. As a consequence, Algorithm 2 performs the internal reduction using the points of this sublattice. In this section, we extend this algorithm to **any** sublattice $\mathcal{L}$ of $\mathcal{L}_B$. This generalised method (we call **GMont-like**) has many properties. We discuss them in this section. Before presenting **GMont-like**, we need to introduce a number of results.

Let $\mathcal{G}$ be a basis of a sublattice $\mathcal{L}$ of $\mathcal{L}_B$. Let us start with the following remark.

Remark 4. Since $\mathcal{L}$ is a sublattice of $\mathcal{L}_B$, there exists a matrix $\mathbf{F} \in \mathbb{Z}^{n \times n}$ such that $\mathcal{G} = \mathbf{F} \times \mathbf{B}$.

We have $\det(\mathbf{B}) = p$ (see Equation 2), so:

$$\det(\mathcal{G}) = tp, \text{ with } t = \det(\mathbf{F}) \in \mathbb{Z} \setminus \{0\}.$$

It is known that $\mathcal{L} = \mathcal{L}_B$ if and only if $\mathbf{F}$ is an unimodular matrix. As a consequence,

$$\mathcal{L} = \mathcal{L}_B \iff \det(\mathcal{G}) = \pm p. \quad (10)$$

For instance, to check whether the matrix $\mathcal{M}$ generates $\mathcal{L}_B$, that is $\mathcal{L}_B = \{ZM \bmod E \mid Z \in \mathbb{Z}_{n-1}[X]\}$, it is sufficient to check if $\det(\mathcal{M}) = \pm p$.

In the PMNS, computations are performed on polynomials in $\mathbb{Z}_{n-1}[X]$ (or equivalently on vectors in $\mathbb{Z}^n$). As a first step to introduce **GMont-like**, we need to characterise $\mathbb{Z}^n$ elements with respect to $\mathcal{G}$.

3.1 Characterising $\mathbb{Z}^n$ elements

The matrix $\mathcal{G}$ is a basis of $\mathbb{R}^n$ (seen as a vector space over $\mathbb{R}$). So, given $V \in \mathbb{R}^n$, there exists $(\alpha_0, \alpha_1, \dots, \alpha_{n-1}) \in \mathbb{R}^n$, such that $V = (\alpha_0, \alpha_1, \dots, \alpha_{n-1})\mathcal{G}$. If $V \in \mathbb{Z}^n$, the coefficients α_i have a special shape, which is given in the proposition below. This shape will be a core element to prove the correctness of **GMont-like** and many other properties.

Proposition 1. *Let $V \in \mathbb{Z}^n$. There exists $(k_0, k_1, \dots, k_{n-1}) \in \mathbb{Z}^n$ such that:*

$$V = \left(\frac{k_0}{d}, \frac{k_1}{d}, \dots, \frac{k_{n-1}}{d} \right) \mathcal{G},$$

where $d = |\det(\mathcal{G})|$.

Proof. The matrix $\mathcal{G}$ is a basis of $\mathbb{R}^n$ (seen as a vector space over $\mathbb{R}$). So, there exists $\beta \in \mathbb{R}^n$, such that $V = \beta \cdot \mathcal{G}$, as $\mathbb{Z}^n \subset \mathbb{R}^n$. Thus, $\beta = V \cdot \mathcal{G}^{-1}$. On the other hand, $\mathcal{G}^{-1} = \frac{1}{\det(\mathcal{G})} \mathbf{G}$, where $\mathbf{G} \in \mathbb{Z}^{n \times n}$ is the adjugate matrix of $\mathcal{G}$. So, $\mathcal{G}^{-1} = \frac{1}{d} \mathbf{J}$, where $\mathbf{J} = \pm \mathbf{G}$ according to the sign of $\det(\mathcal{G})$. Then, $\beta = V \cdot (\frac{1}{d} \mathbf{J}) = \frac{1}{d} K$, where $K = V \cdot \mathbf{J}$. We have $K \in \mathbb{Z}^n$, because $V \in \mathbb{Z}^n$ and $J \in \mathbb{Z}^{n \times n}$. $\square$

From this proposition, we derive the following result.

Corollary 1. *Let $V \in \mathbb{Z}^n$. There exist $(\eta_0, \eta_1, \dots, \eta_{n-1}) \in \mathbb{Z}^n$ and $(\beta_0, \beta_1, \dots, \beta_{n-1}) \in \mathbb{N}^n$ such that:*

$$V = \left(\eta_0 + \frac{\beta_0}{d}, \eta_1 + \frac{\beta_1}{d}, \dots, \eta_{n-1} + \frac{\beta_{n-1}}{d} \right) \mathcal{G},$$

$$= J + S.$$

$$\text{where } \begin{cases} J = (\eta_0, \eta_1, \dots, \eta_{n-1}) \mathcal{G} \in \mathcal{L}, \\ S = (\frac{\beta_0}{d}, \frac{\beta_1}{d}, \dots, \frac{\beta_{n-1}}{d}) \mathcal{G} \in \mathcal{H}. \end{cases}$$

Proof. η_i and β_i are respectively the quotient and the remainder of the Euclidean division of k_i by d . Since $\eta_i \in \mathbb{Z}$, we have $J \in \mathcal{L}$. That is, $J(\gamma) \equiv 0 \pmod{p}$; i.e. $V(\gamma) \equiv S(\gamma) \pmod{p}$. β_i is the remainder of the Euclidean division of k_i by d , so $0 \leq \beta_i < d$. Thus, $0 \leq \frac{\beta_i}{d} < 1$. Hence, $S \in \mathcal{H}$. $\square$

Remark 5. From such a polynomial S , one can easily compute $S' \in \mathcal{H}'$, such that $S'(\gamma) \equiv S(\gamma) \pmod{p}$. In fact, it is sufficient to take $S' = (\alpha_0, \alpha_1, \dots, \alpha_{n-1}) \mathcal{G}$, with:

$$\alpha_i = \begin{cases} \frac{\beta_i}{d} & \text{if } \beta_i < d/2, \\ \frac{\beta_i}{d} - 1 & \text{if not.} \end{cases}$$

The inverse transformation from $\mathcal{H}'$ to $\mathcal{H}$ is also obvious. Thus, $\#(\mathcal{H} \cap \mathbb{Z}^n) = \#(\mathcal{H}' \cap \mathbb{Z}^n)$.

3.2 Montgomery-like internal reduction generalised

This section presents **GMont-like**. As explained in Remark 4, for every basis $\mathcal{G}$ of $\mathcal{L}$, $\det(\mathcal{G}) = tp$, where $t \neq 0$ an integer. Lemma 2 guarantees the existence of an essential parameter for **GMont-like**.

Lemma 2. *Let $\phi \geq 2$ be an integer and $\mathcal{G}$ a basis of $\mathcal{L}$.*

If $\gcd(tp, \phi) = 1$, then $\mathcal{G}' = -\mathcal{G}^{-1} \pmod{\phi}$ exists.

Proof. We have $\mathcal{G}^{-1} = \frac{1}{\det(\mathcal{G})} \mathbf{G} = \frac{1}{tp} \mathbf{G}$, where $\mathbf{G} \in \mathbb{Z}^{n \times n}$ is the adjugate matrix of $\mathcal{G}$. So, $\mathbf{G}^{-1} \pmod{\phi}$ exists. Since $\gcd(tp, \phi) = 1$, $\frac{1}{tp} \pmod{\phi}$ exists as well. $\square$

Algorithm 6 describes **GMont-like**. We assume the parameter ϕ such that $\gcd(tp, \phi) = 1$.

Algorithm 6 Coefficients reduction for PMNS (**GMont-like**)

Require: $C \in \mathbb{Z}^n$, the basis $\mathcal{G}$, $\phi \in \mathbb{N} \setminus \{0, 1\}$ such that $\gcd(tp, \phi) = 1$, and the matrix $\mathcal{G}' = -\mathcal{G}^{-1} \pmod{\phi}$.

Ensure: $S(\gamma) = C(\gamma)\phi^{-1} \pmod{p}$, with $S \in \mathbb{Z}^n$

- 1: $Q = (c_0, \dots, c_{n-1}) \mathcal{G}' \pmod{\phi}$
 - 2: $T = (q_0, \dots, q_{n-1}) \mathcal{G}$
 - 3: $S \leftarrow (C + T)/\phi$
 - 4: return S
-

According to Proposition 1, the input C of Algorithm 6 is such that $C = \alpha \mathcal{G}$, with $\alpha_i = \frac{k_i}{[tp]}$ where $k_i \in \mathbb{Z}$. We have $\gcd(tp, \phi) = 1$, so $\alpha_i \pmod{\phi}$ exists. Thus, the vector

$(-\alpha) \bmod \phi = ((-\alpha_0) \bmod \phi, \dots, (-\alpha_{n-1}) \bmod \phi)$ exists.

Additionally, at line 1 of this algorithm, we have:

$$\begin{aligned} Q &= C\mathcal{G}' \pmod{\phi}, \\ &= (\alpha\mathcal{G})(-\mathcal{G}^{-1}) \pmod{\phi}, \\ &= (-\alpha)(\mathcal{G} \times \mathcal{G}^{-1}) \pmod{\phi}, \\ &= (-\alpha) \bmod \phi. \end{aligned}$$

As a consequence, the output S is such that:

$$S = \frac{\alpha + ((-\alpha) \bmod \phi)}{\phi} \mathcal{G}. \quad (11)$$

The divisions at line 3 in **GMont-like** are thus exact. For these divisions to be fast, one must take ϕ as a power of 2. So, from Lemma 2, we need $\det(\mathcal{G}) \equiv 1 \pmod{2}$; that is, tp should be odd.

As a consequence of Equation 11, if an element is in the fundamental domain $\mathcal{H}$ (see Equation 3), then it remains in $\mathcal{H}$ when **GMont-like** is applied to it.

Property 1. Let $C \in \mathbb{Z}^n$.

If $C \in \mathcal{H}$, then **GMont-like**(C) $\in \mathcal{H}$.

Proof. Assume $C = \alpha\mathcal{G}$. Then, according to Equation 11, **GMont-like**(C) = $\beta\mathcal{G}$, with $\beta_i = \frac{\alpha_i + ((-\alpha_i) \bmod \phi)}{\phi}$.

Remember that $0 \leq (-\alpha_i) \bmod \phi < \phi$. Since $C \in \mathcal{H}$, $0 \leq \alpha_i < 1$. Thus, $0 \leq \beta_i < 1$. Hence, the result. $\square$

Remark 6. Using the idea in [DRV22], with Algorithm 6, the bounds given in Section 2.6.4 become:

$$\rho \geq 2\|\mathcal{G}\|_1 \text{ and } \phi \geq 2w\rho(\delta + 1)^2.$$

However, these bounds can be improved. We will see in Section 5 how this can be done.

3.3 GMont-like and PMNS generation

This generalised Montgomery-like internal reduction greatly simplifies the PMNS generation process. Indeed, the process proposed in [DRV22] to find a suitable polynomial M (which is used to compute $\mathcal{M}$) requires a search in a space of size 2^n . This search is very efficient when n is small, which is most of the time the case for modulus sizes in elliptic curve cryptography. However, as shown in [DRDE], n grows very fast as p becomes large. This makes the search for a suitable polynomial M very expensive. As seen above, **GMont-like** works with any basis $\mathcal{G}$ of any sublattice of $\mathcal{L}_{\mathcal{B}}$. In particular, it works with any (reduced) basis $\mathcal{G}$ of $\mathcal{L}_{\mathcal{B}}$.

As mentioned in Remark 1, such a basis can be obtained with algorithms like LLL [LLL82], BKZ[SE91] or HKZ[KI77], applied to the basis $\mathbf{B}$ (Equation 2). Remember that $\det(\mathbf{B}) = p$, so $\det(\mathcal{G}) = \pm p$. Thus, in this case, ϕ can be taken as a power of 2, assuming p is odd. With this choice, it is no longer necessary to search for a suitable polynomial M .

A possible disadvantage of not using $\mathcal{M}$ for internal reduction could be the memory required to store PMNS parameters. Indeed, the polynomial M is sufficient to compute the parameters $\mathcal{M}$ and $\mathcal{M}'$ since E is known, whereas in the general case the matrix $\mathcal{G}$ is needed for both $\mathcal{G}'$ computation and **GMont-like**. We have $M \in \mathbb{Z}^n$, whereas $\mathcal{G} \in \mathbb{Z}^{n \times n}$.

Remark 7 (An open problem). Being able to choose $\mathcal{G}$ as a basis of any sublattice $\mathcal{L}$ of $\mathcal{L}_{\mathcal{B}}$ gives us the freedom to decide which subset of the polynomials representing zero we want to use to perform coefficient reductions. This could be used to have a "specialised" (and more efficient and/or secure) internal reduction method. For example, it could be an efficient reduction method to use after addition and subtraction. This remains an open problem.

3.4 GMont-like and lattice points

Remember that if $J \in \mathcal{L}$, then $J = \alpha\mathcal{G}$, with $\alpha \in \mathbb{Z}^n$. **GMont-like** behaviour on lattice points is quite interesting. It has many properties we discuss in this section.

Let us begin with the invariants. Some elements of $\mathcal{L}$ are invariant for **GMont-like**. That is, applying **GMont-like** on them returns the input. The following property highlights this situation.

Property 2. Let $J \in \mathcal{L}$, with $J = \alpha\mathcal{G}$.

$$\mathbf{GMont-like}(J) = J \iff \forall i, \alpha_i \in \{0, 1\}.$$

Proof. According to Equation 11, $\mathbf{GMont-like}(J) = (\beta_0, \dots, \beta_{n-1})\mathcal{G}$, where $\beta_i = \frac{\alpha_i + ((-\alpha_i) \bmod \phi)}{\phi}$. Then,

$$\begin{aligned} \mathbf{GMont-like}(J) = J &\iff \beta_i = \alpha_i, \\ &\iff \frac{\alpha_i + ((-\alpha_i) \bmod \phi)}{\phi} = \alpha_i, \\ &\iff (\phi - 1)\alpha_i = ((-\alpha_i) \bmod \phi), \\ &\iff \alpha_i \in \{0, 1\}, \end{aligned}$$

since $0 \leq (-\alpha_i \bmod \phi) < \phi$. □

Notice that a non-lattice point cannot be invariant for **GMont-like**. In fact, such a point represents a non-zero element of $\mathbb{Z}/p\mathbb{Z}$, whereas **GMont-like** induces a factor ϕ^{-1} on the output. In other words: $J \notin \mathcal{L} \Rightarrow \mathbf{GMont-like}(J) \neq J$.

Property 2 leads us to define the **canonical set of zeros**.

Definition 2. Given a basis $\mathcal{G}$ of $\mathcal{L}$, the **canonical set of zeros** $\mathcal{O}$ is defined as follows:

$$\mathcal{O} = \{(\alpha_0, \dots, \alpha_{n-1})\mathcal{G} \mid \alpha_i \in \{0, 1\}\}. \quad (12)$$

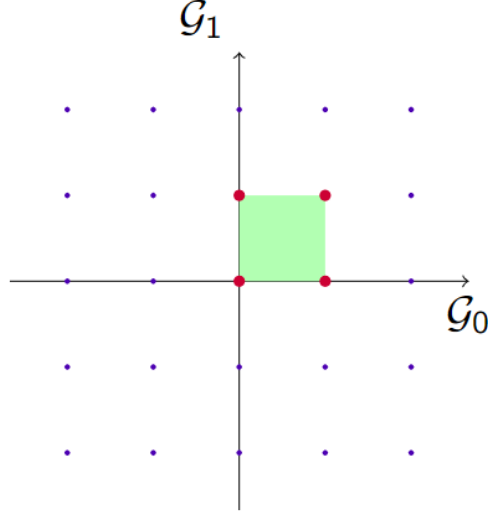
Elements of $\mathcal{O}$ correspond to the points on the edges of the fundamental parallelepiped $\mathcal{H}$. For instance, if $n = 2$, its elements are the points with coordinates $(0, 0)$, $(1, 0)$, $(0, 1)$ and $(1, 1)$ as shown in Figure 1.

Example 1. Let $p = 291791$, a 19-bit prime integer. Let $\mathcal{B} = (p, n, \gamma, \rho, E) = (p, 2, 11810, 841, X^2 - 2)$ be a PMNS, with:

$$\mathcal{G} = \begin{pmatrix} 247 & 420 \\ -593 & 173 \end{pmatrix}.$$

Then, $\mathcal{O} = \{(0, 0), (247, 420), (-593, 173), (-346, 593)\}$.

Note that to avoid constant reminding of parameters, this PMNS will be used in all examples in this paper. In addition, we will avoid showing unnecessary intermediate computations in the examples. Section 8.2 provides a GitHub link where one can find implementations to check all the examples, as well as the intermediate computations.

Figure 1: $\mathcal{O} = \mathcal{H}$ edges

Remark 8. Properties 1 and 2 complement each other quite elegantly. Indeed, one tells us that points on the edges of $\mathcal{H}$ do not change when **GMont-like** is applied to them, while the other tells us that points inside $\mathcal{H}$ stay inside $\mathcal{H}$ when **GMont-like** is applied to them.

Let us now study the effect of **GMont-like** on the output.

Proposition 2. *Let $J \in \mathcal{L}$, with $J = (\alpha_0, \dots, \alpha_{n-1})\mathcal{G}$. If $S = \mathbf{GMont-like}(J)$, then $S = (\beta_0, \dots, \beta_{n-1})\mathcal{G}$, with:*

$$\beta_i = \lceil \frac{\alpha_i}{\phi} \rceil.$$

Proof. Remember that $\alpha_i \in \mathbb{Z}$. According to Equation 11, $\mathbf{GMont-like}(J) = (\beta_0, \dots, \beta_{n-1})\mathcal{G}$, with:

$$\beta_i = \frac{\alpha_i + ((-\alpha_i) \bmod \phi)}{\phi}.$$

The Euclidean division of α_i by ϕ computes q_i and r_i , such that $\alpha_i = q_i\phi + r_i$, with $0 \leq r_i < \phi$ and $q_i = \lfloor \frac{\alpha_i}{\phi} \rfloor$.

Thus, $\beta_i = q_i + \frac{r_i + ((-r_i) \bmod \phi)}{\phi}$.

As a consequence,

- If $\alpha_i \equiv 0 \bmod \phi$, then $r_i = 0$. So, $\beta_i = q_i = \frac{\alpha_i}{\phi} = \lceil \frac{\alpha_i}{\phi} \rceil$.
- If $\alpha_i \not\equiv 0 \bmod \phi$, then $\frac{r_i + ((-r_i) \bmod \phi)}{\phi} = 1$.
Thus, $\beta_i = q_i + 1 = \lceil \frac{\alpha_i}{\phi} \rceil$.

□

A direct consequence of this proposition is a condition for a reduction to $\mathcal{O}$.

Corollary 2. *Let $J \in \mathcal{L}$, with $J = (\alpha_0, \dots, \alpha_{n-1})\mathcal{G}$. If $\forall i \in \{0, \dots, n-1\}$, $-\phi < \alpha_i \leq \phi$, then:*

$$\mathbf{GMont-like}(J) \in \mathcal{O}.$$

Proof. According to Proposition 2, we have the following two cases:

- If $-\phi < \alpha_i \leq 0$, then $\beta_i = 0$.
- If $0 < \alpha_i \leq \phi$, then $\beta_i = 1$.

So, we always have $\beta_i \in \{0, 1\}$. $\square$

Another consequence of Proposition 2 is the following result, which gives a condition for the internal reduction to return the null polynomial. The equality test we present in Section 7 is based on this result.

Corollary 3. *Let $J \in \mathcal{L}$, with $J = (\alpha_0, \dots, \alpha_{n-1})\mathcal{G}$. If $\forall i \in \{0, \dots, n-1\}$, $-\phi < \alpha_i \leq 0$, then:*

$$\mathbf{GMont-like}(J) = 0.$$

Proof. Let us assume that $(\beta_0, \dots, \beta_{n-1})\mathcal{G} = \mathbf{GMont-like}(J)$. According to Proposition 2, we have $\beta_i = 0$, $\forall i \in \{0, \dots, n-1\}$. So, $\mathbf{GMont-like}(J) = (0, \dots, 0)\mathcal{G} = 0$. $\square$

Proposition 2 tells us how **GMont-like** reduces input coefficients. This reduction is such that a number of successive calls to **GMont-like** on an input will result in an element of $\mathcal{O}$. The following result clarifies this number.

Proposition 3. *Let $J \in \mathcal{L}$, with $J = (\alpha_0, \dots, \alpha_{n-1})\mathcal{G}$. Let $k \geq 0$ be the smaller integer such that $\phi^k > \max_i |\alpha_i|$.*

$$\text{If } \tilde{J} = \mathbf{GMont-like}^k(J), \text{ then } \tilde{J} \in \mathcal{O}.$$

Additionally, $\tilde{J} = (\beta_0, \dots, \beta_{n-1})\mathcal{G}$, with:

$$\beta_i = \begin{cases} 0 & \text{if } \alpha_i \leq 0, \\ 1 & \text{if } \alpha_i \geq 1. \end{cases} \quad (13)$$

Proof. Let us assume that $S = (t_0, \dots, t_{n-1})\mathcal{G} = \mathbf{GMont-like}^{k-1}(J)$. Let f be the function defined as follows:

$$\begin{aligned} f &: \mathbb{Z} \rightarrow \mathbb{Z}, \\ u &\mapsto \left\lceil \frac{u}{\phi} \right\rceil. \end{aligned}$$

Thus, $t_i = f^{k-1}(\alpha_i)$, according to Proposition 2.

This function is the composition of the functions $\lceil \cdot \rceil$ and $\frac{\cdot}{\phi}$, which are increasing. In particular, if $u > r\phi$, with $r \in \mathbb{Z}$, then $f(u) > f(r\phi) = r$. Thus, since $\phi^k > \max_i |\alpha_i|$, we have $f^{k-1}(-\phi^k) < t_i \leq f^{k-1}(\phi^k)$. That is,

$$-\phi < t_i \leq \phi.$$

Additionally,

- if $\alpha_i \leq 0$, then $t_i \leq 0$, because $f^{k-1}(\alpha_i) \leq f^{k-1}(0) = 0$,
- if $\alpha_i \geq 1$, then $t_i \geq 1$, because $f^{k-1}(\alpha_i) \geq f^{k-1}(1) = 1$.

We have $\tilde{J} = (\beta_0, \dots, \beta_{n-1})\mathcal{G} = \mathbf{GMont-like}(S)$.

So, $\beta_i = f(t_i) = \left\lceil \frac{t_i}{\phi} \right\rceil$. As a consequence,

- if $\alpha_i \leq 0$, then $-\phi < t_i \leq 0$, so $\beta_i = 0$,
- if $\alpha_i \geq 1$, then $1 \leq t_i \leq \phi$, so $\beta_i = 1$.

Hence, $\tilde{J} \in \mathcal{O}$ and Equation 13. $\square$

According to Property 2, the vector $\tilde{J}$ computed in Proposition 3 is invariant for **GMont-like**. We define this vector as the **canonical representation** of J .

This proposition provides two ways to compute the canonical representation $\tilde{J}$ of a lattice point $J = \alpha\mathcal{G}$. Indeed, if α is known, Equation 13 gives a very simple way to compute $\tilde{J}$. However, if only a bound on $\|\alpha\|_\infty$ (equals to $\max_i |\alpha_i|$) is known, i.e. smaller than an integer w , this proposition tells us that:

$$\tilde{J} = \mathbf{GMont-like}^k(J),$$

with $k = \lceil \log_\phi(w) \rceil$.

With **GMont-like** and some of the properties presented in this section, it is possible to further reduce the coefficients of elements in the PMNS. The next section introduces some additional properties that we will use in Section 5 to perform internal reduction in some special domains that allow to control the redundancy and to reduce memory requirement in the PMNS.

4 Reductions to some special domains

As seen in Lemma 1, each element $a \in \mathbb{Z}/p\mathbb{Z}$ has at least one representation in the domains $\mathcal{H}$ and $\mathcal{H}'$ (see Equations 3 and 4). With Remarks 2 and 6, it can be observed that reducing PMNS elements to these domains would further reduce the bound on the parameter ρ . Additionally, we will need the reduction to $\mathcal{H}$ to study the redundancy.

In this section, based on **GMont-like**, we present required properties for coefficient reductions in $\mathcal{H}$ and $\mathcal{H}'$. Before that, we need to introduce the domain $\mathcal{D}_j$, a set that will help us both to improve coefficients bound and to control the redundancy in the PMNS.

4.1 The domain $\mathcal{D}_j$

It is a subset of $\mathbb{R}^n$ that can be seen as an extension of the fundamental region $\mathcal{H}'$.

Definition 3. Let $j \geq 1$ be an integer. Given a basis $\mathcal{G}$, the domain $\mathcal{D}_j$ is defined as follows:

$$\mathcal{D}_j = \left\{ \sum_{i=0}^{n-1} \mu_i \mathcal{G}_i \mid -j \leq \mu_i < j, \text{ with } \mu_i \in \mathbb{R} \right\}. \quad (14)$$

Two direct consequences of this definition are the following properties.

Property 3.

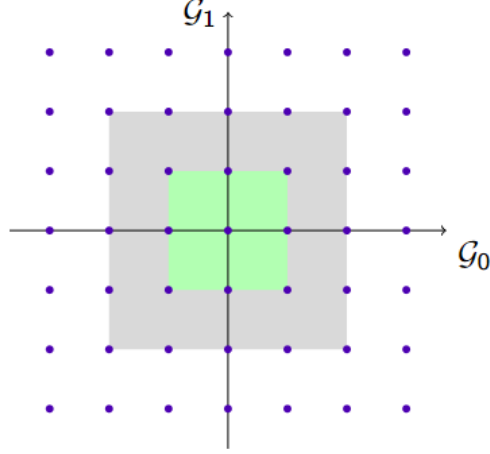
- If $i < j$, then $\mathcal{D}_i \subset \mathcal{D}_j$.
- If $A \in \mathcal{D}_i$ and $B \in \mathcal{D}_j$, then $A + B \in \mathcal{D}_{i+j}$.

Example 2. Let us take $n = 2$. Figure 2 shows the domain $\mathcal{D}_1$ inside the domain $\mathcal{D}_2$.

Notice that if $A \in \mathcal{D}_j$, then $\|A\|_\infty \leq j\|\mathcal{G}\|_1$. In particular, if $A \in \mathcal{D}_1$, then $\|A\|_\infty \leq \|\mathcal{G}\|_1$. With the bounds given in Remark 6, **GMont-like** performs reductions to a domain $\mathcal{D}_j$ with $j \geq 2$, since $\rho \geq 2\|\mathcal{G}\|_1$. If we could do the reduction to $\mathcal{D}_1$, it would allow to take $\rho = \|\mathcal{G}\|_1 + 1$, thus reducing the memory required to represent PMNS elements. This section explains how to do this.

Let us first see Lemma 3 which ensures that this value for ρ allows to define a PMNS. Then, we explain how to perform the reduction to $\mathcal{D}_1$.

This straightforward result extends Theorem 1 for any sublattice of $\mathcal{L}_{\mathcal{B}}$.

Figure 2: Domain $\mathcal{D}_1$ inside $\mathcal{D}_2$

Lemma 3. Let $\mathcal{L}$ be a sublattice of $\mathcal{L}_{\mathcal{B}}$, having $\mathcal{G}$ as a basis. A tuple $\mathcal{B} = (p, n, \gamma, \rho)$ defines a PMNS if:

$$\rho > \frac{1}{2} \|\mathcal{G}\|_1.$$

Proof. According to Lemma 1, each element $a \in \mathbb{Z}/p\mathbb{Z}$ has at least one representation $A \in \mathcal{H}'$. Additionally, $A \in \mathcal{H}'$ implies that $\|A\|_{\infty} \leq \frac{1}{2} \|\mathcal{G}\|_1$ (see Remark 2). $\square$

According to this lemma, taking $\rho = \frac{1}{2} \|\mathcal{G}\|_1 + 1$ allows to build a PMNS. Since coefficients in this system can be negative, it contains at most $(\|\mathcal{G}\|_1 + 1)^n$ polynomials. Each element $a \in \mathbb{Z}/p\mathbb{Z}$ has at least one representation in the PMNS, thus one must have $p \leq (\|\mathcal{G}\|_1 + 1)^n$. The following property, which is a direct consequence of Hadamard's inequality, shows that this is the case.

Property 4.

$$|\det(\mathcal{G})| \leq (\|\mathcal{G}\|_1)^n.$$

Proof. Let $c_0, c_1, \dots, c_{n-1}$ be the columns of $\mathcal{G}$. According to Hadamard's inequality, we have

$$|\det(\mathcal{G})| \leq \prod_{i=0}^{n-1} \|c_i\|_2.$$

Moreover, we have $\|c_i\|_2 \leq \|c_i\|_1 \leq \|\mathcal{G}\|_1$. $\square$

As mentioned in Remark 4, $\det(\mathcal{G}) = tp$, with $t \in \mathbb{Z} \setminus \{0\}$, since it is a basis of the sublattice $\mathcal{L}$. So, $p \leq |\det(\mathcal{G})|$. Thus,

$$p \leq |\det(\mathcal{G})| \leq (\|\mathcal{G}\|_1)^n < (\|\mathcal{G}\|_1 + 1)^n.$$

Now, let us consider the following result which gives a condition for a reduction to $\mathcal{D}_1$.

Proposition 4. Let $A \in \mathbb{Z}^n$, with $A = (\alpha_0, \dots, \alpha_{n-1})\mathcal{G}$. If $\forall i \in \{0, \dots, n-1\}$, $-\phi \leq \alpha_i \leq 0$, then,

$$\mathbf{GMont-like}(A) \in \mathcal{D}_1.$$

Proof. Assume $S = (\beta_0, \dots, \beta_{n-1})\mathcal{G} = \mathbf{GMont-like}(J)$. According to Equation 11, we have

$$\beta_i = \frac{\alpha_i + ((-\alpha_i) \bmod \phi)}{\phi}.$$

Since $0 \leq (-\alpha_i) \bmod \phi < \phi$ and $-\phi \leq \alpha_i \leq 0$,

$$-\phi \leq \alpha_i + ((-\alpha_i) \bmod \phi) < \phi.$$

Thus, $-1 \leq \beta_i < 1$. Hence, the result. $\square$

Note that Proposition 4 requires that the vector α be such that $-\phi \leq \alpha_i \leq 0$, $\forall i \in \{0, \dots, n-1\}$. To satisfy this requirement, we introduce the **translation vector**.

4.2 The translation vector

Let $\mathcal{B} = (p, n, \gamma, \rho, E)$ be a PMNS. This section first introduces the translation vector $\mathcal{T}$ along with some of its properties.

As mentioned above, the goal is to satisfy the requirement (given in Proposition 4) on the input for an internal reduction in $\mathcal{D}_1$, in order to have PMNS elements in $\mathcal{D}_1$. To do this, we will use $\mathcal{T}$ to translate some $\mathbb{Z}^n$ elements to a region in $\mathbb{R}^n$ where they have negative coordinates with respect to the basis $\mathcal{G}$, without changing the values which are represented. This translation is meant to be done in the conversions (to PMNS) and the modular multiplication for outputs in $\mathcal{D}_1$. So, we define the translation vector $\mathcal{T}$ with respect to these operations.

Let us first consider the modular multiplication. Let $A, B \in \mathbb{Z}^n$ be the results of adding at most $\delta + 1$ elements of $\mathcal{B}$; so, we have $\|A\|_\infty, \|B\|_\infty \leq (\delta + 1)(\rho - 1)$.

Property 5. Let $C = (A \times B) \bmod E$.
Then, $C = \alpha \mathcal{G}$ with $\alpha \in \mathbb{R}^n$ such that

$$\|\alpha\|_\infty \leq w(\delta + 1)^2(\rho - 1)^2 \|\mathcal{G}^{-1}\|_1.$$

Proof. The matrix $\mathcal{G}$ is also a basis of $\mathbb{R}^n$ (seen as a vector space over $\mathbb{R}$). So, there exists $\alpha \in \mathbb{R}^n$ such that $C = \alpha \mathcal{G}$. On the one hand, according to Equation 7, we have:

$$\|C\|_\infty \leq w(\delta + 1)^2(\rho - 1)^2.$$

On the other hand, we have $\alpha = C\mathcal{G}^{-1}$. So,

$$\begin{aligned} \|\alpha\|_\infty &= \|C\mathcal{G}^{-1}\|_\infty, \\ &\leq \|C\|_\infty \|\mathcal{G}^{-1}\|_1, \\ &\leq w(\delta + 1)^2(\rho - 1)^2 \|\mathcal{G}^{-1}\|_1. \end{aligned}$$

$\square$

Let us now focus on the conversion process, more precisely the fast conversion to PMNS (Algorithm 5). This algorithm requires the polynomials P_i , where each P_i represents $\rho^i \phi^2$. It performs a radix- ρ decomposition, where ρ is assumed to be a power of two for efficiency. Since we want PMNS elements in $\mathcal{D}_1$, we take $\rho = \|\mathcal{G}\|_1 + 1$. Such ρ is unlikely to be a power of two. So, to keep the conversion fast, we need to redefine the polynomials P_i .

Remember that p is the modulus for which the PMNS is built. Let k and β be two integers such that

$$k = \left\lceil \frac{\log_2(p)}{n} \right\rceil \quad \text{and} \quad \beta = 2^k. \quad (15)$$

Then, $p \leq \beta^n$. As a consequence, a radix- β decomposition on n coefficients of any $a \in \mathbb{Z}/p\mathbb{Z}$ is always possible. The polynomials P_i are now computed as representations of $\beta^i \phi^2$, i.e.

$$P_i(\gamma) \equiv (\beta^i \phi^2) \pmod{p}. \quad (16)$$

As a consequence, the polynomial U computed at line 2 of Algorithm 5 is such that

$$\|U\|_\infty \leq n(\beta - 1)(\rho - 1).$$

As with Property 5 for modular multiplication, this implies that

$$U = \alpha\mathcal{G} \text{ with } \|\alpha\|_\infty \leq n(\beta - 1)(\rho - 1)\|\mathcal{G}^{-1}\|_1. \quad (17)$$

We can now define the translation vector $\mathcal{T}$.

Definition 4. Let m and u be two integers such that

$$m = \max(n(\beta - 1), w(\delta + 1)^2(\rho - 1)),$$

and

$$u = \lceil m(\rho - 1)\|\mathcal{G}^{-1}\|_1 \rceil. \quad (18)$$

The **translation vector** $\mathcal{T}$ is defined as

$$\mathcal{T} = (-u, \dots, -u)\mathcal{G}. \quad (19)$$

Example 3. In our example of PMNS, $\rho = 841$. The parameter $E(X) = X^2 - 2$. So, $w = 3$.

$$\mathcal{G}^{-1} = \frac{1}{291791} \begin{pmatrix} 173 & -420 \\ 593 & 247 \end{pmatrix}.$$

With $\delta = 0$, we have $u = 5557$.
So, $\mathcal{T} = (-u, -u)\mathcal{G} = (1922722, -3295301)$.

A consequence of Definition 4 is the following result.

Property 6. Let $V \in \mathbb{Z}^n$ be a vector. If V satisfies

$$\|V\|_\infty \leq (\rho - 1) \max(n(\beta - 1), w(\delta + 1)^2(\rho - 1)),$$

then,

$$V + \mathcal{T} = (\beta_0, \beta_1, \dots, \beta_{n-1})\mathcal{G},$$

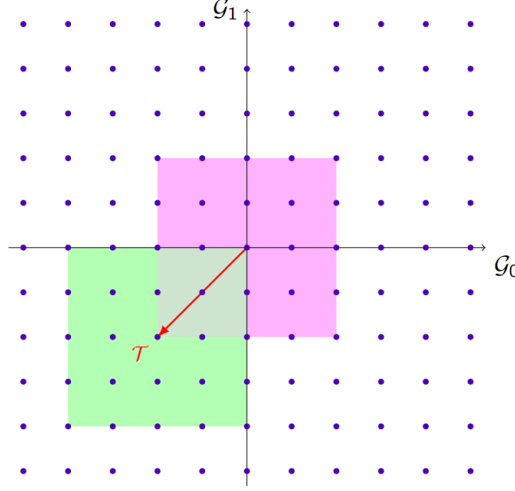
with $-2u \leq \beta_i \leq 0$.

Proof. This directly comes from how $\mathcal{T}$ is computed. Indeed, as seen with Property 5 and Equation 17, we have $V = \alpha\mathcal{G}$, where $\|\alpha\|_\infty \leq m(\rho - 1)\|\mathcal{G}^{-1}\|_1 \leq u$. That is, $-u \leq \alpha_i \leq u$. Since $\beta_i = \alpha_i - u$, one can conclude. $\square$

Notice that $\mathcal{T} \in \mathcal{L}$, since $u \in \mathbb{N}$. So, $\mathcal{T}(\gamma) \equiv 0 \pmod{p}$. Thus, V and $V + \mathcal{T}$ represent the same element. That is, the translation with $\mathcal{T}$ does not change the value which is represented.

Example 4. Let us take $n = 2$ and $u = 2$. Figure 3 shows $\mathcal{T}$ effect on elements in $\mathcal{D}_2$.

With the translation vector $\mathcal{T}$, we can now discuss the reductions in $\mathcal{D}_1$, $\mathcal{H}$ and then in $\mathcal{H}'$.

Figure 3: Domain translation with $\mathcal{T}$

4.3 Reduction to $\mathcal{D}_1$

Proposition 4 requires that the input A coordinates be negative, with respect to the base $\mathcal{G}$. Thanks to the translation vector and the right bound on ϕ , we can relax this constraint to allow positive coordinates.

Proposition 5. *Let $A \in \mathbb{Z}^n$, with $A = (\alpha_0, \dots, \alpha_{n-1})\mathcal{G}$. Let us assume that $\phi \geq 2u$. If $\forall i \in \{0, \dots, n-1\}$, $-u \leq \alpha_i \leq u$, then*

$$\mathbf{GMont-like}(A + \mathcal{T}) \in \mathcal{D}_1.$$

Proof. We have $A + \mathcal{T} = \beta\mathcal{G}$, with $-\phi \leq -2u \leq \beta_i \leq 0$, since $\phi \geq 2u$. Proposition 4 allows to conclude. $\square$

This proposition has the following consequence.

Corollary 4. *Let us assume that $\phi \geq 2u$. If $V \in \mathbb{Z}^n$ is a vector such that*

$$\|V\|_\infty \leq (\rho - 1) \max(n(\beta - 1), w(\delta + 1)^2(\rho - 1)),$$

then

$$\mathbf{GMont-like}(V + \mathcal{T}) \in \mathcal{D}_1.$$

Proof. From Equation 18, $V = (\alpha_0, \dots, \alpha_{n-1})\mathcal{G}$, with $-u \leq \alpha_i \leq u$. Proposition 5 allows to conclude. $\square$

This corollary gives a condition on the input that ensures the conversion and multiplication outputs are in $\text{lat}\mathcal{D}_1$. In Section 5.1, we will see how to modify the corresponding algorithms to take advantage of this result. Let us now see how to perform the reduction to the fundamental regions $\mathcal{H}$ and $\mathcal{H}'$.

4.4 Reduction to $\mathcal{H}$

Recall that the fundamental region $\mathcal{H}$ is defined as

$$\mathcal{H} = \{t \in \mathbb{R}^n \mid t = \sum_{i=0}^{n-1} \mu_i \mathcal{G}_i \text{ and } 0 \leq \mu_i < 1\}.$$

The reduction to $\mathcal{H}$ is done on an element already in $\mathcal{D}_1$. So, we assume in this section that

$$\phi \geq 2u.$$

The reduction to $\mathcal{H}$ is based on the following result.

Proposition 6. *Let $A \in \mathcal{D}_1 \cap \mathbb{Z}^n$.*

If k is the smallest integer such that $\phi^k > \det(\mathcal{G})$, then

$$\mathbf{GMont-like}^k(A) \in \mathcal{H}.$$

Proof. Let $d = |\det(\mathcal{G})|$. Let us assume that $A = \alpha\mathcal{G}$ and consider the following notation:

$$A^{(j+1)} = \mathbf{GMont-like}(A^{(j)}).$$

So, $A^{(j)} = \mathbf{GMont-like}^{(j)}(A)$, with $A = A^{(0)}$.

As already mentioned, we have $A^{(j+1)} = \alpha^{(j+1)}\mathcal{G}$, with

$$\alpha_i^{(j+1)} = \frac{\alpha_i^{(j)} + ((-\alpha_i^{(j)}) \bmod \phi)}{\phi}. \quad (20)$$

Notice that if $\alpha_i^{(j)} \in [0, 1)$, then $\alpha_i^{(j+1)} \in [0, 1)$.

Since $A \in \mathcal{D}_1$, we have $-1 \leq \alpha_i^{(j)} < 1$. Thus, if $\alpha_i^{(j)} \not\equiv 0 \pmod{\phi}$, then $\alpha_i^{(j+1)} \in [0, 1)$, according to Equation 20.

We have $A^{(0)} = A \in \mathbb{Z}^n$, so $\alpha_i^{(0)} = \frac{k_i^{(0)}}{d}$, with $k_i^{(0)} \in \mathbb{Z}$, according to Proposition 1.

Remember that $\gcd(d, \phi) = 1$ for **GMont-like** (see Section 3.2). Thus, if $\alpha_i^{(j)} \equiv 0 \pmod{\phi}$, then $k_i^{(j)} \equiv 0 \pmod{\phi}$ and $\alpha_i^{(j+1)} = \frac{\alpha_i^{(j)}}{\phi}$. Since $\phi^k > d$, it implies that $-\phi^k < k_i^{(0)} < \phi^k$. As a consequence, $\forall i \in \{0, \dots, n-1\}$, there exists a positive integer $t_i < k$ such that $\alpha_i^{(t_i)} \not\equiv 0 \pmod{\phi}$. With Equation 20, it leads to $\alpha_i^{(t_i+1)} \in [0, 1)$. So, $\alpha_i^k \in [0, 1)$, since $k \geq t_i + 1$. $\square$

A consequence of this proposition is the following result, which is due to u expression.

Corollary 5. *Let $A \in \mathcal{D}_1 \cap \mathbb{Z}^n$. Then,*

$$\mathbf{GMont-like}^n(A) \in \mathcal{H}.$$

Proof. According to Property 4, $\det(\mathcal{G}) \leq (\|\mathcal{G}\|_1)^n$. Since $\phi > \rho > \|\mathcal{G}\|_1$, we have $\phi^n > \det(\mathcal{G})$. Additionally, if $A \in \mathcal{H}$, then $\mathbf{GMont-like}(A) \in \mathcal{H}$. So, one can conclude using Proposition 6. $\square$

4.5 Reduction to $\mathcal{H}'$

Recall that the fundamental region $\mathcal{H}'$ is defined as

$$\mathcal{H}' = \{t \in \mathbb{R}^n \mid t = \sum_{i=0}^{n-1} \mu_i \mathcal{G}_i \text{ and } -\frac{1}{2} \leq \mu_i < \frac{1}{2}\}.$$

The main interest of this reduction is to further reduce the memory required to represent PMNS elements, since $A \in \mathcal{H}'$ implies that $\|A\|_\infty \leq \frac{1}{2}\|\mathcal{G}\|_1$. This means that the theoretical optimal bound is obtained (see Lemma 3).

The reduction to $\mathcal{H}'$ is done on an element already in $\mathcal{H}$. So, we also assume in this section that

$$\phi \geq 2u.$$

The main idea of this reduction is the following. Let $A \in \mathcal{H}$, with $A = (\alpha_0, \dots, \alpha_{n-1})\mathcal{G}$. The polynomial $A' \in \mathcal{H}'$ representing the same element is such that $A' = (\alpha'_0, \dots, \alpha'_{n-1})\mathcal{G}$, where

$$\alpha'_i = \begin{cases} \alpha_i & \text{if } \alpha_i < 1/2, \\ \alpha_i - 1 & \text{else.} \end{cases}$$

GMont-like (Algorithm 6) can be slightly modified to directly apply this idea during the internal reduction. The idea is to compute the centralised remainder for the coefficients of Q (at line 1 of this algorithm). That is, instead of having $0 \leq q_i < \phi$, we take $-\frac{\phi}{2} \leq q_i < \frac{\phi}{2}$ by computing $q_i - \phi$ if $q_i \geq \frac{\phi}{2}$. This operation is notated as $(\text{mod}^\pm \phi)$. Algorithm 7 incorporates the centralised remainder. From now on, we will refer to it as **SMont-like** (i.e. Signed **GMont-like**).

Remark 9. If ϕ is a power of two, as it should be for **GMont-like** to be efficient, computing the centralised remainder is very cheap and does not involve any conditional branching. It is a very simple binary operation. Let us assume that $\phi = 2^h$, with $h \in \mathbb{N} \setminus \{0\}$. Given $q_i \in \mathbb{Z}/\phi\mathbb{Z}$, we have $q_i (\text{mod}^\pm \phi) = q_i - \phi(q_i \gg (h-1))$. The symbol $\gg$ denotes the binary right shift operator, i.e. $q_i \gg (h-1) = \lfloor \frac{q_i}{2^{h-1}} \rfloor$. Note that since $q_i \in \mathbb{Z}/\phi\mathbb{Z}$, we have $q_i \gg (h-1) \in \{0, 1\}$. So its multiplication by ϕ has no cost. Using the PMNS generation process proposed in [DDV20, DRV22], computing the centralised remainder has no additional cost, as it only implies the use of signed (instead of unsigned) variables to store the coefficients of Q . So, we assume no additional cost for this operation, since we always use this generation process with $\phi = 2^h$.

Algorithm 7 Signed coefficients reduction (**SMont-like**)

Require: $A \in \mathbb{Z}^n$, $\phi = 2^h$, and the matrices $\mathcal{G}$ and $\mathcal{G}'$.

Ensure: $S(\gamma) = A(\gamma)\phi^{-1} (\text{mod } p)$, with $S \in \mathbb{Z}^n$

- 1: $Q = (a_0, \dots, a_{n-1})\mathcal{G}' (\text{mod}^\pm \phi)$ # centralised remainder for each q_i
 - 2: $T = (q_0, \dots, q_{n-1})\mathcal{G}$
 - 3: $S \leftarrow (A + T)/\phi$
 - 4: return S
-

Property 7. If $A \in \mathcal{H} \cap \mathbb{Z}^n$, then

$$\mathbf{SMont-like}(A) \in \mathcal{H}'.$$

Proof. Remember that we take $\phi = 2^h$. Let us assume that $A = \alpha\mathcal{G}$. From Equation 11, the vector Q at line 1 is such that $q_i = (-\alpha_i) (\text{mod}^\pm \phi)$. So, the output S is such that $S = \beta\mathcal{G}$, with

$$\beta_i = \frac{\alpha_i + ((-\alpha_i) \text{mod}^\pm \phi)}{\phi}.$$

Since $A \in \mathcal{H}$, we have $0 \leq \alpha_i < 1$. Since $((-\alpha_i) \text{mod}^\pm \phi)$ is an integer, we also have $-\frac{\phi}{2} \leq ((-\alpha_i) \text{mod}^\pm \phi) \leq \frac{\phi}{2} - 1$. Thus, $-\frac{1}{2} \leq \beta_i < \frac{1}{2}$. Hence, $S \in \mathcal{H}'$, with $\|S\|_\infty \leq \frac{1}{2}\|\mathcal{G}\|_1$. $\square$

In the previous section, we saw how to compute a representation in $\mathcal{H}$. With Property 7, this gives the following result.

Proposition 7. Let $A \in \mathcal{D}_1 \cap \mathbb{Z}^n$. Then,

$$\mathbf{SMont-like}(\mathbf{GMont-like}^n(A)) \in \mathcal{H}'.$$

Proof. Let $R = \mathbf{GMont-like}^n(A)$. According to Corollary 5, $R \in \mathcal{H}$. Hence, from Property 7, $\mathbf{SMont-like}(R) \in \mathcal{H}'$. $\square$

With Sections 3 and 4, we introduced a number of theoretical tools to better understand the behaviour of elements in the PMNS. In the following sections, we explain how to use these tools

- for smaller coefficients size (Section 5),
- to study the redundancy (Section 6),
- to perform the equality test in the PMNS (Section 7).

5 Internal reduction in $\mathcal{D}_1$, $\mathcal{H}$ and $\mathcal{H}'$

In the previous section, we introduced the domain $\mathcal{D}_j$. We explained that the reduction to $\mathcal{D}_1$ would allow the bound on ρ to be reduced from $2\|\mathcal{G}\|_1$ to $\|\mathcal{G}\|_1$, thus allowing to take $\rho = \|\mathcal{G}\|_1 + 1$. We also presented results to do so. In addition, we introduced properties for reduction in the domains $\mathcal{H}$ and $\mathcal{H}'$. The main interest of these domains is to study redundancy in the PMNS, as we will see in Section 6.

To have PMNS elements in $\mathcal{D}_1$, we need the conversion (to PMNS) and the multiplication algorithms to output the results in $\mathcal{D}_1$. In this section, we first see how to do this. Then, given $A \in \mathcal{B}$, we show how to compute an equivalent representation of A in $\mathcal{H}$ and $\mathcal{H}'$.

5.1 Internal reduction to $\mathcal{D}_1$

In order to have PMNS elements in $\mathcal{D}_1$, the internal reduction to $\mathcal{D}_1$ must to be done during the modular multiplication (Algorithm 3) and the conversions in the PMNS (Algorithms 4 and 5). Bellow, we present a slight modification of these algorithms, using the translation vector $\mathcal{T}$.

Algorithm 9 requires the polynomials $P_i \in \mathcal{D}_1$ which are representations of $\beta^i \phi^2$ (see Equation 15 for β). These polynomials are computed using Algorithm 8. Proposition 8 ensures an output in $\mathcal{D}_1$ for Algorithms 8 and 9. Notice that Algorithm 8 is also faster than Algorithm 4, since it makes one less call to **GMont-like**. Remark that the precomputed data τ is $\phi^{n-1} \bmod p$, instead of $\phi^n \bmod p$. We remind that **GMont-like** is Algorithm 6, described in Section 3.2.

Algorithm 8 Exact conversion to PMNS with translation

Require: $a \in \mathbb{Z}/p\mathbb{Z}$, $\mathcal{B} = (p, n, \gamma, \rho, E)$, $\tau = \phi^{n-1} \bmod p$, the matrices $\mathcal{G}$, $\mathcal{G}'$ and the translation vector $\mathcal{T}$.

Ensure: $A \in \mathcal{D}_1$, such that $A \equiv a_{\mathcal{B}}$

- 1: $\alpha = a \times \tau \pmod{p}$
 - 2: $U = (\alpha, 0, \dots, 0)$ # a polynomial of degree 0
 - 3: **for** $i = 0 \dots n - 3$ **do**
 - 4: $U \leftarrow \mathbf{GMont-like}(U)$
 - 5: **end for**
 - 6: $V \leftarrow U + \mathcal{T}$
 - 7: $A \leftarrow \mathbf{GMont-like}(V)$
 - 8: **return** A
-

Proposition 8. *If ρ and ϕ are such that*

$$\rho > \|\mathcal{G}\|_1 \quad \text{and} \quad \phi \geq 2u,$$

then, with $a \in \mathbb{Z}/p\mathbb{Z}$ as input, Algorithms 8 and 9 output a polynomial $A \in \mathcal{D}_1$.

Algorithm 9 Fast conversion to PMNS with translation

Require: $a \in \mathbb{Z}/p\mathbb{Z}$, $\mathcal{B} = (p, n, \gamma, \rho, E)$, $P_i \in \mathcal{D}_1$ such that $P_i(\gamma) \equiv (\beta^i \phi^2) \pmod{p}$ and the translation vector $\mathcal{T}$.

Ensure: $A \in \mathcal{D}_1$, such that $A \equiv (a\phi)_{\mathcal{B}}$

- 1: $t = (t_{n-1}, \dots, t_0)_{\beta}$ # radix- β decomposition of a
- 2: $U \leftarrow \sum_{i=0}^{n-1} t_i P_i(X)$
- 3: $W \leftarrow U + \mathcal{T}$
- 4: $A \leftarrow \mathbf{GMont-like}(W)$
- 5: return A

Proof. Let us start with Algorithm 8. Let $\beta \in \mathbb{R}^n$ be the vector such that $U = \beta\mathcal{G}$ (line 2). We have $\beta = U\mathcal{G}^{-1}$, so $\|\beta\|_{\infty} \leq \|U\|_{\infty} \|\mathcal{G}^{-1}\|_1 < p \|\mathcal{G}^{-1}\|_1 \leq (\|\mathcal{G}\|_1)^n \|\mathcal{G}^{-1}\|_1$, since $\alpha < p$ (line 1) and $p \leq (\|\mathcal{G}\|_1)^n$ according to Property 4.

Also, since $\delta \geq 0$, $\rho > \|\mathcal{G}\|_1$ and $w \geq n \geq 2$, we have $\phi \geq 4(\|\mathcal{G}\|_1)^2 \|\mathcal{G}^{-1}\|_1$. So, the vector U after the first round of the loop at line 3 is such that $U = \beta\mathcal{G}$, with $\|\beta\|_{\infty} < (\|\mathcal{G}\|_1)^{n-2}$. Notice that this loop is executed only if $n \geq 3$.

Remember that $\|\mathcal{G}\|_1 \|\mathcal{G}^{-1}\|_1 \geq 1$, so $\phi \geq 4\|\mathcal{G}\|_1$. Thus, after the next $n-3$ rounds of this loop, U will be such that $\|\beta\|_{\infty} < \|\mathcal{G}\|_1 < u$. So, according to Property 6, the vector V (at line 6) is such that $V = (\nu_0, \nu_1, \dots, \nu_{n-1})\mathcal{G}$, with $-2u \leq \nu_i \leq 0$. Since $\phi \geq 2u$, we have $-\phi \leq \nu_i \leq 0$. Thus, Proposition 4 ensures that the output $A \in \mathcal{D}_1$.

Let us now consider Algorithm 9. According to Equation 17, the polynomial U (line 2) is such that $\|U\|_{\infty} \leq n(\beta-1)(\rho-1)$. Since $\phi \geq 2u$, the output $A \in \mathcal{D}_1$, according to Corollary 4. $\square$

Algorithm 10 Modular multiplication with translation (**TransMult**)

Require: $A, B \in \mathbb{Z}_{n-1}[X]$, the matrices $\mathcal{G}, \mathcal{G}', \mathcal{E}$, $\phi \in \mathbb{N} \setminus \{0, 1\}$ and the translation vector $\mathcal{T}$.

Ensure: $S(\gamma) = A \cdot B(\gamma)\phi^{-1} \pmod{p}$, with $S \in \mathbb{Z}_{n-1}[X]$

- 1: $C = A \times B$
- 2: $V = (c_0, \dots, c_{n-1}) + (c_n, \dots, c_{2n-2})\mathcal{E}$
- 3: $R = V + \mathcal{T}$
- 4: $S \leftarrow \mathbf{GMont-like}(R)$
- 5: return S

Modular multiplication is applied to inputs that are the result of adding at most $\delta + 1$ elements of $\mathcal{B}$. The following proposition gives the bounds for the output to be in $\mathcal{D}_1$.

Proposition 9. *Let $A, B \in \mathbb{Z}_{n-1}[X]$ be two polynomials, such that: $\|A\|_{\infty}, \|B\|_{\infty} \leq (\delta + 1)(\rho - 1)$. If ϕ satisfies the inequality*

$$\phi \geq 2u \text{ (see Equation 18),}$$

then, with A and B as inputs, Algorithm 10 outputs a polynomial $S \in \mathcal{D}_1$.

Proof. The vector V computed at line 2 of this algorithm is such that $V = A \times B \bmod E$. So, $\|V\|_{\infty} \leq w(\delta + 1)^2(\rho - 1)^2$. Since $\phi \geq 2u$, Corollary 4 ensures that $S \in \mathcal{D}_1$. $\square$

Remark 10. Some observations about these reductions to $\mathcal{D}_1$ are the following.

- Since the conversions only require that $\rho > \|\mathcal{G}\|_1$, we will take

$$\rho = \|\mathcal{G}\|_1 + 1 \text{ and } \phi \geq 2u. \quad (21)$$

- The additional cost of reducing in $\mathcal{D}_1$ is the addition of the translation vector $\mathcal{T}$. This cost is very small compared to that of **GMont-like**; see Section 8, where operation costs are discussed.

With these algorithms and bounds, $\mathcal{B}$ elements will be in $\mathcal{D}_1$. Thus, to study the redundancy in $\mathcal{B}$, we will study the redundancy in $\mathcal{D}_1$. Before studying this redundancy, let us see how to compute a representation of a given element in the fundamental regions $\mathcal{H}$ and $\mathcal{H}'$.

5.2 Internal reduction to $\mathcal{H}$

Recall the fundamental region

$$\mathcal{H} = \{t \in \mathbb{R}^n \mid t = \sum_{i=0}^{n-1} \mu_i \mathcal{G}_i \text{ and } 0 \leq \mu_i < 1\}.$$

In this section, based on Corollary 5, we present an algorithm to compute representations in the fundamental domain $\mathcal{H}$.

We consider the bounds on ρ and ϕ , given in Remark 10. Corollary 5 requires a polynomial $A \in \mathcal{D}_1$. Also, remember that **GMont-like** introduces a factor ϕ^{-1} on the output. So, to compute a representation in $\mathcal{H}$ of an element $A \in \mathcal{B}$, we use a polynomial $\nu \in \mathcal{D}_1$ which represents ϕ^{n+1} , i.e. $\nu(\gamma) \equiv \phi^{n+1} \pmod{p}$. It can be precomputed using Algorithm 8 or 9. With such a polynomial, a representation in $\mathcal{H}$ can be computed using Algorithm 11.

Algorithm 11 Representation computation in $\mathcal{H}$

Require: $A \in \mathcal{D}_1$, the polynomial $\nu \in \mathcal{D}_1$, such that $\nu(\gamma) \equiv \phi^{n+1} \pmod{p}$.

Ensure: $S \in \mathcal{H}$ and $A(\gamma) \equiv S(\gamma) \pmod{p}$

- 1: $S \leftarrow \text{TransMult}(A, \nu)$ # Algorithm 10
 - 2: **for** $i = 0 \dots n - 1$ **do**
 - 3: $S \leftarrow \text{GMont-like}(S)$
 - 4: **end for**
 - 5: **return** S
-

According to Proposition 9, the vector S (line 1, Algorithm 11) belongs to $\mathcal{D}_1$. Thus, at the end of the loop (line 4), we have $S \in \mathcal{H}$, according to Corollary 5. We have $\mathcal{T} \in \mathcal{L}$, i.e. $\mathcal{T}(\gamma) \equiv 0 \pmod{p}$. Since **GMont-like** induces a factor ϕ^{-1} on the output and $\nu(\gamma) \equiv \phi^{n+1} \pmod{p}$, it is clear that $A(\gamma) \equiv S(\gamma) \pmod{p}$.

Finally, to compute a representation of an element $a \in \mathbb{Z}/p\mathbb{Z}$ in $\mathcal{H}$, one first computes a representation $A \in \mathcal{D}_1$ (using Algorithm 8 or 9) and then applies Algorithm 11 on A .

5.3 Internal reduction to $\mathcal{H}'$

Recall the fundamental region

$$\mathcal{H}' = \{t \in \mathbb{R}^n \mid t = \sum_{i=0}^{n-1} \mu_i \mathcal{G}_i \text{ and } -\frac{1}{2} \leq \mu_i < \frac{1}{2}\}.$$

As mentioned in Section 4.5, the main interest of this reduction is to further reduce the memory required to represent PMNS elements, since $A \in \mathcal{H}'$ implies that $\|A\|_\infty \leq \frac{1}{2}\|\mathcal{G}\|_1$. That is, the theoretical optimal bound (given in Lemma 3) is obtained. Here also, we consider the bound on ρ and ϕ , given in Remark 10. Additionally, remember that for

SMont-like (Algorithm 7, Section 4.5), we assume that ϕ is a power of two such that $\phi = 2^h$, which is the best case for efficiency.

Proposition 7 requires a polynomial $A \in \mathcal{D}_1$. Like **GMont-like**, **SMont-like** induces a factor ϕ^{-1} on the output. To compute a representation in $\mathcal{H}'$ of an element $A \in \mathcal{B}$, we first precompute the polynomial $\xi \in \mathcal{B}$ representing ϕ^{n+2} . Then, Algorithm 12 can be applied.

Algorithm 12 Representation computation in $\mathcal{H}'$

Require: $A \in \mathcal{D}_1$, the polynomial $\xi \in \mathcal{D}_1$, such that $\xi(\gamma) \equiv \phi^{n+2} \pmod{p}$.

Ensure: $S \in \mathcal{H}'$ and $A(\gamma) \equiv S(\gamma) \pmod{p}$

```

1:  $R \leftarrow \text{TransMult}(A, \xi)$  # Algorithm 10
2: for  $i = 0 \dots n-1$  do
3:    $R \leftarrow \text{GMont-like}(R)$ 
4: end for
5:  $S \leftarrow \text{SMont-like}(R)$ 
6: return  $S$ 

```

According to Proposition 9, the vector R (line 1, Algorithm 11) is such that $R \in \mathcal{D}_1$. Hence, $S \in \mathcal{H}'$, according to Proposition 7. We have $\mathcal{T}(\gamma) \equiv 0 \pmod{p}$. Since **GMont-like** and **SMont-like** induce a factor ϕ^{-1} on the output and $\xi(\gamma) \equiv \phi^{n+2} \pmod{p}$, it is clear that $A(\gamma) \equiv S(\gamma) \pmod{p}$.

Finally, to compute a representation of an element $a \in \mathbb{Z}/p\mathbb{Z}$ in $\mathcal{H}'$, one first computes a representation $A \in \mathcal{D}_1$ (using Algorithm 8 or 9) and then apply Algorithm 12 on A .

To summarise this section, we have explained how to compute a representation in $\mathcal{D}_1$, $\mathcal{H}$ or $\mathcal{H}'$ of an element $a \in \mathbb{Z}/p\mathbb{Z}$. We also studied how to do modular multiplication for an output in $\mathcal{D}_1$. Let us now study the redundancy in these sets.

6 Redundancy in the PMNS

This section is about the redundancy in $\mathcal{H}$, $\mathcal{H}'$ and $\mathcal{D}_j$ (which includes $\mathcal{D}_1$). We first discuss the number of representations in these sets. Then, we explain how to compute these representations.

6.1 Number of representations

Some elements in $\mathbb{Z}/p\mathbb{Z}$ can have more than one representation in $\mathcal{H} \cap \mathbb{Z}^n$ (or $\mathcal{H}' \cap \mathbb{Z}^n$). Indeed, according to Lemma 1, every element $a \in \mathbb{Z}/p\mathbb{Z}$ has **at least** one representation in these sets. This is not precise enough for an exact study of redundancy. The following result gives the condition for representation uniqueness in these sets.

Property 8. If $\mathcal{L} = \mathcal{L}_{\mathcal{B}}$, then each $a \in \mathbb{Z}/p\mathbb{Z}$ has exactly one representation in $\mathcal{H} \cap \mathbb{Z}^n$, and $\#(\mathcal{H} \cap \mathbb{Z}^n) = p$.

Proof. From Lemma 1, we know that a has at least one representation in $\mathcal{H} \cap \mathbb{Z}^n$. Let $A, A' \in \mathcal{H} \cap \mathbb{Z}^n$ such that: $a = A(\gamma) \pmod{p} = A'(\gamma) \pmod{p}$. So, $(A - A')(\gamma) \equiv 0 \pmod{p}$. That is, $A - A' \in \mathcal{L}_{\mathcal{B}}$. Since $A, A' \in \mathcal{H}$, we have $A - A' = (\nu_0, \dots, \nu_{n-1})\mathcal{G}$, with $-1 < \nu_i < 1$ and $\nu_i \in \mathbb{N}$ as $A - A' \in \mathcal{L}_{\mathcal{B}}$. Thus, $\forall i, \nu_i = 0$. As a consequence, $A - A' = 0$, i.e. $A = A'$. So, each $a \in \mathbb{Z}/p\mathbb{Z}$ has exactly one representation in $\mathcal{H} \cap \mathbb{Z}^n$. We have $\#(\mathcal{H} \cap \mathbb{Z}^n) \geq p$. If $\#(\mathcal{H} \cap \mathbb{Z}^n) > p$, then at least one element $a \in \mathbb{Z}/p\mathbb{Z}$ would necessarily have more than one representation, which is impossible. Hence, $\#(\mathcal{H} \cap \mathbb{Z}^n) = p$. $\square$

A consequence of this property is also the uniqueness of representation $\mathcal{H}' \cap \mathbb{Z}^n$, with $\#(\mathcal{H}' \cap \mathbb{Z}^n) = p$, if $\mathcal{L} = \mathcal{L}_{\mathcal{B}}$. This comes from Remark 5.

Remark 11. $\mathcal{L} = \mathcal{L}_{\mathcal{B}}$ means that $\mathcal{G}$ is a basis of $\mathcal{L}_{\mathcal{B}}$. As mentioned in Remark 4, this is equivalent to having $\det(\mathcal{G}) = \pm p$. For example, for the classical Montgomery-like internal reduction method (Algorithm 2), one has to check if $\det(\mathcal{M}) = \pm p$ (see Equation 9 for $\mathcal{M}$).

Let us now discuss the redundancy in the domain $\mathcal{D}_j$. This set has an interesting connection with the fundamental region $\mathcal{H}$. This is highlighted in the following result.

Property 9. Let $j \geq 1$ be an integer.

The set $\mathcal{D}_j$ contains exactly $(2j)^n$ times the set $\mathcal{H}$.

Proof. Let $\mathcal{G}_i$ be the i -th row of $\mathcal{G}$. We respectively define $\mathcal{H}^{(i)}$ and $\mathcal{D}_j^{(i)}$ as the fundamental region and the domain j with respect to the i -th coordinate, that is

$$\mathcal{D}_j^{(i)} = \{\mu \mathcal{G}_i \mid -j \leq \mu < j, \text{ with } \mu \in \mathbb{R}\},$$

and

$$\mathcal{H}^{(i)} = \{\mu \mathcal{G}_i \mid 0 \leq \mu < 1, \text{ with } \mu \in \mathbb{R}\}.$$

Thus, we have

$$\mathcal{D}_j^{(i)} = \bigcup_{t \in \mathbb{Z} \cap [-j, j)} \{t \mathcal{G}_i + \mathcal{H}^{(i)}\}.$$

That is, $\mathcal{D}_j^{(i)}$ is the union of $\mathcal{H}^{(i)}$ translations by the vectors $t \mathcal{G}_i$, where $t \in \mathbb{Z} \cap [-j, j)$.

If $t_1 \neq t_2$, with $t_1, t_2 \in \mathbb{Z}$, then $\{t_1 \mathcal{G}_i + \mathcal{H}^{(i)}\} \cap \{t_2 \mathcal{G}_i + \mathcal{H}^{(i)}\} = \emptyset$. So, each domain $\mathcal{D}_j^{(i)}$ contains exactly $2j$ times $\mathcal{H}^{(i)}$.

$\mathcal{D}_j$ is a direct sum of $\mathcal{D}_j^{(i)}$. More precisely, we have

$$\mathcal{D}_j = \mathcal{D}_j^{(0)} \oplus \mathcal{D}_j^{(1)} \oplus \dots \oplus \mathcal{D}_j^{(n-1)},$$

We also have

$$\mathcal{H} = \mathcal{H}^{(0)} \oplus \mathcal{H}^{(1)} \oplus \dots \oplus \mathcal{H}^{(n-1)}.$$

Thus, $\mathcal{D}_j$ contains exactly $\underbrace{(2j) * (2j) * \dots * (2j)}_{n \text{ times}}$ times the fundamental domain $\mathcal{H}$. $\square$

Given $a \in \mathbb{Z}/p\mathbb{Z}$, we define $\mathcal{R}_j(a)$ as the set of polynomials that represent a in $\mathcal{D}_j$:

$$\mathcal{R}_j(a) = \{A \in \mathcal{D}_j \cap \mathbb{Z}^n \mid a = A(\gamma) \pmod{p}\}. \quad (22)$$

The set $\mathcal{R}_j(0)$ is quite easy to compute. Indeed, $\mathcal{R}_j(0) \subset \mathcal{L}_{\mathcal{B}}$. Since $\mathcal{R}_j(0) \subset \mathcal{D}_j$, we have

$$\mathcal{R}_j(0) = \{(\alpha_0, \dots, \alpha_{n-1}) \mathcal{G}, \text{ with } \alpha_i \in \mathbb{Z} \cap [-j, j)\}. \quad (23)$$

The following result gives $\mathcal{R}_j(a)$ cardinality and defines it with respect to $\mathcal{R}_j(0)$.

Proposition 10. Let us assume that $\mathcal{L} = \mathcal{L}_{\mathcal{B}}$.

Let $a \in \mathbb{Z}/p\mathbb{Z}$. If A is its unique representation in $\mathcal{H} \cap \mathbb{Z}^n$, then

$$\mathcal{R}_j(a) = \{A + J \mid J \in \mathcal{R}_j(0)\}, \quad (24)$$

and

$$\#\mathcal{R}_j(a) = (2j)^n. \quad (25)$$

Proof. According to Property 8, a has exactly one representation in $\mathcal{H}$. Since $\mathcal{D}_j$ contains exactly $(2j)^n$ times the set $\mathcal{H}$ (according to Property 9), we deduce that $\#\mathcal{R}_j(a) = (2j)^n$. In particular, $\#\mathcal{R}_j(0) = (2j)^n$. So, the set $\mathcal{K} = \{A + J \mid J \in \mathcal{R}_j(0)\}$ is such that $\#\mathcal{K} = (2j)^n$. We have $\mathcal{K} \subset \mathcal{D}_j$. Since $B \in \mathcal{K} \Rightarrow a = B(\gamma) \pmod{p}$, it implies that $\mathcal{K} \subset \mathcal{R}_j(a)$. As a consequence, $\mathcal{R}_j(a) = \mathcal{K}$, because $\#\mathcal{K} = \#\mathcal{R}_j(a) = (2j)^n$. $\square$

According to this proposition, **if $\mathcal{L} = \mathcal{L}_{\mathcal{B}}$, then each element $a \in \mathbb{Z}/p\mathbb{Z}$ has exactly $(2j)^n$ representations in $\mathcal{D}_j$.**

So, each element $a \in \mathbb{Z}/p\mathbb{Z}$ has exactly 2^n representations in the PMNS, with the internal reduction in $\mathcal{D}_1$ (discussed in Section 5.1). In comparison, using the approach presented in [DDV20, DRV22] where the internal reduction is done in $\mathcal{D}_j$ with $j \geq 2$ (see Section 4.1), each element $a \in \mathbb{Z}/p\mathbb{Z}$ has at least 4^n representations in the PMNS.

6.2 Representation computations

Let us now see how to compute $\mathcal{R}_j(a)$, the set of all representations in $\mathcal{D}_j$ of an element $a \in \mathbb{Z}/p\mathbb{Z}$.

We assume that $\mathcal{L} = \mathcal{L}_{\mathcal{B}}$. With Equation 23, it is easy to pre-compute $\mathcal{R}_j(0)$. To compute the representations of a in $\mathcal{D}_j$, one first computes its unique representation in $\mathcal{H}$, using Algorithm 11. Then, $\mathcal{R}_j(a)$ can be computed using Equation 24.

The following example shows how to compute all representations of an element in $\mathcal{D}_1$.

Example 5. Let us go back to our example of PMNS. We have $p = 291791 = \det(\mathcal{G})$, so $\mathcal{L} = \mathcal{L}_{\mathcal{B}}$.

$$\mathcal{R}_1(0) = \{-593X + 346, -173X + 593, -420X - 247, 0\}$$

The unique representation of $a = 122706$ in $\mathcal{H}$ is $A(X) = 381X - 39$, with

$$(-39, 381) = \left(\frac{219186}{291791}, \frac{110487}{291791}\right)\mathcal{G}.$$

So, $\mathcal{R}_1(a) = \{-212X + 307, 208X + 554, -39X - 286, 381X - 39\}$. Its unique representation in $\mathcal{H}'$ is $-39X - 286$, with

$$(-286, -39) = \left(\frac{-72605}{291791}, \frac{110487}{291791}\right)\mathcal{G}.$$

We have now studied redundancy in the PMNS, with a process to compute the representations in $\mathcal{D}_j$ of a given element in $\mathbb{Z}/p\mathbb{Z}$. The next section discusses equality test in the PMNS.

7 Equality test in the PMNS

Many cryptographic applications [FS07, AVL19, BOS, Joy19] require to perform arithmetic equality test. This operation is however non-trivial in the PMNS, due to its redundancy property. Indeed, since elements can have more than one representation, a simple comparison of two polynomials is not sufficient to conclude that they represent two different elements in $\mathbb{Z}/p\mathbb{Z}$. Until now, the best way to know if two polynomials $A, B \in \mathbb{Z}_{n-1}[X]$ represent the same element, i.e. $A(\gamma) \equiv B(\gamma) \pmod{p}$, is to first compute $C = A - B$, then evaluate C modulo p to check if $C(\gamma) \equiv 0 \pmod{p}$. So, a conversion out of the PMNS (the evaluation of C) is required. Such a conversion is not appropriate for some cryptographic implementation, for instance when comparison are used to protect against fault attacks [BOS]. In this section, we present a method to perform equality test on

elements in the PMNS. This method does not require any evaluation. Its cost is discussed in Section 8.1.

The equality test is based on Corollary 3. This corollary requires the coordinates of J , with respect to the basis $\mathcal{G}$, to be negative. To fulfill this requirement, we use the translation vector $\mathcal{T}$. Note that having A and B represent the same element is equivalent to having $A - B \in \mathcal{L}$.

Let $\ell \in \mathbb{N}$ be an integer such that:

$$\ell = \frac{1}{2}w(\delta + 1)^2(\rho - 1)^2. \quad (26)$$

Algorithm 13 performs the equality test between two polynomials $A, B \in \mathbb{Z}_{n-1}[X]$ such that $\|A\|_\infty, \|B\|_\infty < \ell$. The bounds on ρ and ϕ remain the same as those given in Remark 10.

Algorithm 13 Equality test in the PMNS

Require: $A, B \in \mathbb{Z}_{n-1}[X]$ with $\|A\|_\infty, \|B\|_\infty < \ell$, and the translation vector $\mathcal{T}$.

Ensure: $S = 0$ if and only if $A(\gamma) \equiv B(\gamma) \pmod{p}$

- 1: $C = (A - B) + \mathcal{T}$
 - 2: $S \leftarrow \mathbf{GMont-like}(C)$
 - 3: return S
-

Proposition 11. *Let $A, B \in \mathbb{Z}_{n-1}[X]$ be two polynomials, such that $\|A\|_\infty, \|B\|_\infty < \ell$. Then, Algorithm 13 outputs the null polynomial if and only if $A - B \in \mathcal{L}$.*

Proof. Notice that the output S is such that $S = \mathbf{GMont-like}(A - B + \mathcal{T})$.

Let us first assume that $S = 0$. Since $\mathcal{T} \in \mathcal{L}$, we thus have $\mathbf{GMont-like}(A - B) \equiv 0$. That is, $\frac{A-B}{\phi} \in \mathcal{L}$. Hence, $A - B \in \mathcal{L}$.

Let us now assume that $A - B \in \mathcal{L}$. So, $A - B + \mathcal{T} \in \mathcal{L}$. Since $\|A\|_\infty, \|B\|_\infty < \frac{1}{2}w(\delta + 1)^2(\rho - 1)^2$, we have $\|A - B\|_\infty < w(\delta + 1)^2(\rho - 1)^2$. So, as seen in Property 5 proof, $A - B = \alpha\mathcal{G}$, with $\|\alpha\|_\infty < w(\delta + 1)^2(\rho - 1)^2\|\mathcal{G}^{-1}\|_1 \leq u$. Thus, $A - B + \mathcal{T} = \beta\mathcal{G}$, with $-2u < \beta_i < 2u$. Since $\phi \geq 2u$, one can conclude that $\mathbf{GMont-like}(A - B + \mathcal{T}) = 0$, according to Corollary 3. $\square$

Remark 12. The bound ℓ (Equation 26) on the infinity norms of the inputs is quite large. Indeed, since $w \geq 2$ and $\delta \geq 0$, we have $\ell \geq (\rho - 1)^2$. Remember that if $A \in \mathcal{B}$, then $\|A\|_\infty \leq (\rho - 1)$. Thus, Algorithm 13 applies to elements in the PMNS.

Example 6. As seen in the previous example, $u = 5557$ and $\mathcal{T} = (1922722, -3295301)$. Let us take $\phi = 2^{16}$, thus

$$\mathcal{G}' = \begin{pmatrix} 59709 & 63772 \\ 61473 & 7591 \end{pmatrix}.$$

Let $A(X) = 50X + 623$ and $B(X) = -197X - 217$, be two representations of 7541. Let $C(X) = 55X - 3$, be a representation of 65965.

One can check that $\mathbf{GMont-like}(A - B + \mathcal{T}) = 0$, whereas $\mathbf{GMont-like}(A - C + \mathcal{T}) = 372X - 178$.

The code provided in Section 8.2 can be used to check these results.

8 Operation costs

In this section, we discuss the theoretical cost of the main algorithms presented in this paper. We also provide a link to a SageMath implementation to generate PMNS, to study the redundancy and to check the equality test.

8.1 Theoretical costs

This section is about the costs of the main algorithms presented in this paper. We use the notations and conventions adopted in [DRV22]. We assume a k -bit processor architecture. So, the basic arithmetic computations are performed on k -bit words. We also assume that the algorithms inputs belong to a PMNS $\mathcal{B}$, such that $\phi = 2^h$, where $h \in \mathbb{N}$ and $1 \leq h \leq k$. Thus, an element in $\mathcal{B}$ requires nk bits to be represented.

The symbols $\mathcal{M}$ and $\mathcal{A}$ respectively denote the multiplication and the addition of two k -bit integers. Also, $\mathcal{S}_l^i$ and $\mathcal{S}_r^i$ respectively denote a left shift and a right shift of i bits.

We have $\|\mathcal{T}\|_\infty \leq u\|\mathcal{G}\|_1 < \phi^2 \leq 2^{2k}$. So, adding $\mathcal{T}$ to an element costs $2n\mathcal{A}$.

Table 1 gives the costs of the main algorithms. It is based on the costs given in [DRV22] (Section 8.1). The modular multiplication method costs are based on the efficient polynomials E presented in Table 1 of that paper. For the fast conversion algorithms, we do not take into account the radix- β decomposition of the input. Assuming that β is a power of two (as we suggested, see Equation 15), this decomposition can be done very efficiently.

Table 1: Main algorithm costs for $\phi = 2^h$

Algorithm	Cost
Mont-like (Algorithm 2) [DRV22]	$2n^2\mathcal{M} + (3n^2 - n)\mathcal{A} + n\mathcal{S}_r^h$
MulMod (Algorithm 3) [DRV22]	$3n^2\mathcal{M} + (5n^2 - n - 2)\mathcal{A} + n\mathcal{S}_r^h$
FastConv (Algorithm 5) [DRV22]	$3n^2\mathcal{M} + (5n^2 - 3n)\mathcal{A} + n\mathcal{S}_r^h$
GMont-like (Algorithm 6)	$2n^2\mathcal{M} + (3n^2 - n)\mathcal{A} + n\mathcal{S}_r^h$
SMont-like (Algorithm 7)	$2n^2\mathcal{M} + (3n^2 - n)\mathcal{A} + n\mathcal{S}_r^h$
Mult to $\mathcal{D}_1$ (Algorithm 10)	$3n^2\mathcal{M} + (5n^2 + n - 2)\mathcal{A} + n\mathcal{S}_r^h$
Conv to $\mathcal{D}_1$ (Algorithm 9)	$3n^2\mathcal{M} + (5n^2 - n)\mathcal{A} + n\mathcal{S}_r^h$
Equality test (Algorithm 13)	$2n^2\mathcal{M} + (3n^2 + 3n)\mathcal{A} + n\mathcal{S}_r^h$

It can be observed in this table that the modular multiplication costs are similar to the one of the classical Montgomery modular multiplication. In [DDV20] (Section 6.4) and [DRV22] (Section 8.2), C implementations of PMNS are compared with implementations of the classical Montgomery modular multiplication in GMP [Ga] and OpenSSL [Pro] libraries, for both performance and memory requirement. They appear to perform similarly. In [DDMG25], it is shown that PMNS can be an interesting alternative for modular arithmetic in elliptic curve cryptography. Note that these PMNS implementations do not take advantage of the high parallelisation capability of this system. As explained in Remark 9, the centralised remainder has no additional cost. This is why **GMont-like** and **SMont-like** have the same cost. Finally, it can be seen that an equality test in the PMNS costs almost an internal reduction.

8.2 Some codes for testing

Using SageMath library [Sa], we have implemented the methods and algorithms presented in this paper. These implementations are available on GitHub:

<https://github.com/arithPMNS/PMNS-and-redundancy>

Given a prime p , one can generate PMNS, with the choice of parameters such as n , δ and z . Given a PMNS, this code allows to compute the representation we want for any $a \in \mathbb{Z}/p\mathbb{Z}$. It also allows to perform equality check. This code can be used to check the examples presented in this paper.

9 Elliptic Curve Diffie–Hellman with PMNS

Elliptic Curve Diffie–Hellman (ECDH) is a key agreement protocol that allows two parties to establish a shared secret over an insecure channel. Normally, using PMNS for ECDH would require conversion out of the system at the end of the process to ensure that both parties have exactly the same secret. As an application of the work presented in this paper, we show in this section how PMNS can replace classical arithmetic in ECDH while always remaining in the system, i.e. without requiring conversion out of the PMNS.

In this section, we consider elliptic curves in the short Weierstrass form. Of course, this can be applied in a similar way to any other elliptic curve. Note that we focus here on the basic ECDH protocol, without any countermeasures against attacks such as man-in-the-middle attacks. These countermeasures can be integrated as easily as when using the classical representation for modular arithmetic. As an example, we apply this protocol to the BrainpoolP256r1 curve.

As practical validation, we have implemented the work presented in this section using the SageMath library. It is available on GitHub at the following link:

<https://github.com/arithPMNS/ECDH-with-PMNS>

In Section 9.5, we compare our C implementation with ECDH implementations using OpenSSL, for the BrainpoolP256r1, BrainpoolP320r1 and BrainpoolP384r1 curves. These implementations are also available on GitHub:

<https://github.com/arithPMNS/ECDH-using-PMNS-and-OpenSSL>

9.1 Modular inversion in the PMNS

Before presenting the ECDH protocol with the PMNS, we need to discuss the modular inversion in the PMNS. In his PhD thesis [Dos20], Dosso adapts some modular exponentiation methods to the PMNS. For example, we have Algorithm 14, which is an adaptation of the Montgomery ladder modular exponentiation method [JY02] to the PMNS.

Let Θ be a representation of 1 in the PMNS. It is computed using Algorithm 9, which also puts the result in the Montgomery domain. That is, $\Theta(\gamma) \equiv (\phi) \pmod{p}$. Of course, Θ can be computed directly using Algorithm 8 with ϕ as input. Algorithm 14 requires Θ .

Let $k \in \mathbb{N}$ be an integer. Assuming that its bit size is less than $\ell \in \mathbb{N}$, i.e. $\lfloor \log_2(k) \rfloor + 1 \leq \ell$, $(k_{\ell-1}, \dots, k_0)_2$ denotes the big-endian binary representation of k on ℓ bits. This means that the execution of Algorithm 14 is in constant time, i.e. regardless of the binary decomposition of k , there will be ℓ iterations in the For loop of this algorithm.

Algorithm 14 Montgomery ladder in the PMNS [Dos20] (**ModExpo**)

Require: $A \in \mathbb{Z}_{n-1}[X]$ such that $A(\gamma) \equiv (a\phi) \pmod{p}$, $k = (k_{\ell-1}, \dots, k_0)_2$, $\mathcal{B} = (p, n, \gamma, \rho, E)$, and $\Theta \in \mathcal{B}$ such that $\Theta(\gamma) \equiv (\phi) \pmod{p}$

Ensure: $R_0 \in \mathbb{Z}_{n-1}[X]$, with $R_0(\gamma) \equiv (a^k \phi) \pmod{p}$

```

1:  $R_0 \leftarrow \Theta$ 
2:  $R_1 \leftarrow A$ 
3: for  $i = \ell - 1$  to 0 do
4:    $b \leftarrow k_i$ 
5:    $R_{1-b} \leftarrow \text{TransMult}(R_{1-b}, R_b)$  # Algorithm 10
6:    $R_b \leftarrow \text{TransMult}(R_b, R_b)$ 
7: end for
8: return  $R_0$ 

```

Let $a \in \mathbb{Z}/p\mathbb{Z} \setminus \{0\}$ and b be its inverse in $\mathbb{Z}/p\mathbb{Z}$, i.e. $b = a^{-1} \pmod{p}$. The inverse of a in the PMNS is a polynomial B such that $B(\gamma) \equiv (b\phi) \pmod{p}$.

Remember that p is a prime number, so according to Fermat's little theorem, we have $b = a^{p-2} \pmod{p}$. As a consequence, to compute the inverse of an element (which does not represent zero) in the PMNS, we use Algorithm 14 with $k = p - 2$. Algorithm 15 performs this operation.

Algorithm 15 Modular inversion in the PMNS [Dos20] (**ModInv**)

Require: $A \in \mathbb{Z}_{n-1}[X]$ such that $A(\gamma) \equiv (a\phi) \pmod{p}$ with $a \neq 0$, and $\mathcal{B} = (p, n, \gamma, \rho, E)$

Ensure: $B \in \mathbb{Z}_{n-1}[X]$, with $B(\gamma) \equiv (a^{-1}\phi) \pmod{p}$

- 1: $k \leftarrow p - 2$
 - 2: $B \leftarrow \mathbf{ModExpo}(A, k)$ # Algorithm 14
 - 3: return B
-

9.2 From affine to Jacobian coordinates and vice versa

Let $E(\mathbb{F}_p)$ be an elliptic curve in the short Weierstrass form with equation

$$E(\mathbb{F}_p) : y^2 = x^3 + ax + b \quad (27)$$

A point on $E(\mathbb{F}_p)$ is defined by its (x, y) affine coordinates. Using affine coordinates for curve arithmetic implies modular inversions, which are quite expensive compared to other modular operations. To avoid such inversions, projective coordinate systems have been introduced. One of the most interesting is the Jacobian coordinate system.

In the Jacobian coordinates, Equation 27 becomes

$$E(\mathbb{F}_p) : Y^2 = X^3 + aXZ^4 + bZ^6 \quad (28)$$

A point is defined by its (X, Y, Z) Jacobian coordinates. In [BL], formulae for point doubling and addition are given.

Remark 13. In the Jacobian coordinate system, the representation of points is not unique, regardless of the number system used. This means that two points (X, Y, Z) and (X', Y', Z') can represent the same point. To find the unique representation of a point, we need to go back to its affine coordinates. Given a point $G(X, Y, Z)$ in Jacobian coordinates (with $Z \neq 0$), the corresponding affine point is $G(x, y)$ with

$$x = \frac{X}{Z^2}, \quad y = \frac{Y}{Z^3}. \quad (29)$$

Given an affine point $G(x, y)$, a corresponding Jacobian point is $G(x, y, 1)$.

The ECDH protocol requires several elliptic curve scalar multiplications (ECSM). It is the operation kG , where the scalar k is an integer and G is a point on a curve $E(\mathbb{F}_p)$. This operation can be done in several ways. Many algorithms are proposed in [GJM⁺11, JY02]. To perform the ECSM with the PMNS, we first convert the coordinates of G in the PMNS. Then, any scalar multiplication algorithm can be used with G (in the PMNS) and the scalar k . For our implementation of the ECSM (with PMNS), we chose the Montgomery ladder method [JY02].

Remark 14 (From Jacobian to affine coordinates in the PMNS). Let $G \in E(\mathbb{F}_p)$ be a Jacobian point with its coordinates in PMNS. That is, $G = (G_X, G_Y, G_Z)$ where

$$G_i(\gamma) \equiv (i\phi) \pmod{p}.$$

Note that the factor ϕ comes from the conversion in the Montgomery domain, which is necessary for consistent use of **GMont-like**, as explained earlier.

Let us assume that G is not the point at infinity, i.e. $0 \neq G_Z(\gamma) \pmod{p}$. To compute the corresponding affine point in the PMNS (based on Equation 29), we first compute the inverse of G_Z using Algorithm 15. Let us call this inverse B . Then, using Algorithm 10, we compute the affine coordinates (G_x, G_y) with $G_x = G_X \times B^2$ and $G_y = G_Y \times B^3$.

Remark 15 (From affine to Jacobian coordinates in the PMNS). Let $G \in E(\mathbb{F}_p)$ be an affine point with its coordinates in PMNS. That is, $G = (G_x, G_y)$ where

$$G_i(\gamma) \equiv (i\phi) \pmod{p}.$$

A corresponding Jacobian point is $G = (G_x, G_y, \Theta)$, where $\Theta \in \mathcal{B}$ is a representation of 1, i.e. such that $\Theta(\gamma) \equiv (\phi) \pmod{p}$, see Section 9.1.

Remark 16. As mentioned above, the point representation is unique in affine coordinates. Note, however, that if the number system used to represent the coordinates is redundant, as is the case with PMNS, then two affine points can actually be the same point. This must be taken into account in redundant number systems such as PMNS. This is an important element in the ECDH protocol that we present below.

9.3 ECDH using PMNS

Using the ECDH protocol with PMNS consists of two main steps. The first step is the setup, which consists of converting the parameters of the curves and the coordinates of the base point in the PMNS. The second step is the actual ECDH protocol, where two parties share a secret. The first main step consists of the following points.

1. Generate a PMNS for the modulus of the curve, as explained in Section 3.3. The parameter δ must be chosen according to the point addition formulas to be used.
2. Convert the curve parameters to the PMNS. We consider elliptic curves in the short Weierstrass form. So, we need to convert a and b in the PMNS. This is done using Algorithm 9, which also makes a conversion to the Montgomery domain. Let A and B be the computed representations of a and b respectively. So, $A(\gamma) \equiv (a\phi) \pmod{p}$ and $B(\gamma) \equiv (b\phi) \pmod{p}$.
3. Convert the affine coordinates of the base point $G(x, y)$ in the PMNS, also using Algorithm 9. We notate $(G_{_x}, G_{_y})$ the result of this conversion.

Note that this first step only needs to be done once. These parameters in the PMNS are the ones to publish, along with the order q of the base point G , which can be edited directly in the Public Key Certificates (PKC).

Now suppose Alice and Bob want to use the ECDH protocol to share a common secret in a public space. They both have access to the above public data. The steps of the ECDH protocol are the following.

1. For safety, both Alice and Bob check that the point G actually belongs to the elliptic curve $E(\mathbb{F}_p)$ to be safe against attacks such as the small subgroup attacks [LL97]. Elements are in the PMNS. So, they compute in the PMNS $L(X) = G_{_y}(X)^2$ and $R(X) = G_{_x}(X)^3 + A(X) * G_{_x}(X) + B(X)$. Then, they use Algorithm 13 to check if L and R represent the same element. If this is not the case, the protocol must stop here.
2. For efficient curve arithmetic, we use the Jacobian coordinates (you can choose another one if you prefer). Remember that q is the order of the base point G . Alice randomly chooses an integer $\tilde{a} \in [1, q)$. This is her secret. She then performs the elliptic curve scalar multiplication (ECSM) $Q_{\tilde{A}} = \tilde{a}G$. Section 9.2 explains

how to do this in the PMNS. Alice converts $Q_{\tilde{A}}$ in affine coordinates (Remark 14 explains how to do this in the PMNS). Using Algorithm 11, she computes the unique representations of the affine coordinates of $Q_{\tilde{A}}$ in $\mathcal{H}$, and forwards the resulting data to Bob. Bob does the same on his side. That is, he randomly chooses his secret $\tilde{b} \in [1, q)$, computes $Q_{\tilde{B}} = \tilde{b}G$, converts it in affine coordinates (in $\mathcal{H}$) and sends it to Alice.

Note: Neither Alice nor Bob has to convert their public points to affine coordinates (in $\mathcal{H}$) before sending them. The protocol works with public points sent in Jacobian coordinates. However, we propose converting them to avoid side-channel attacks that exploit redundancy in the public points (see, for example, Naccache et al. [NSS04]).

3. Now, Alice has her secret $\tilde{a}$ and the point $Q_{\tilde{B}}$, while Bob has his secret $\tilde{b}$ and the point $Q_{\tilde{A}}$. Alice checks whether $Q_{\tilde{B}}$ belongs to the elliptic curve $E(\mathbb{F}_p)$, using Algorithm 13 and the equation of $E(\mathbb{F}_p)$. If $Q_{\tilde{B}}$ does indeed belong to $E(\mathbb{F}_p)$, Alice performs the ECSM $S_{\tilde{A}} = \tilde{a}Q_{\tilde{B}}$. In ECDH, the common secret is the affine coordinate x . So, she computes the affine coordinate $\tilde{A}_{-x}$ of $S_{\tilde{A}}$, as explained in Remark 14. Bob does the same check on $Q_{\tilde{A}}$. He computes $S_{\tilde{B}} = \tilde{b}Q_{\tilde{A}}$, then computes the affine coordinates $\tilde{B}_{-x}$ of $S_{\tilde{B}}$.
4. Note that: $S_{\tilde{A}} = (\tilde{A}_{-x}, \tilde{A}_{-y}) = \tilde{a}Q_{\tilde{B}} = (\tilde{a}\tilde{b})G$ and $S_{\tilde{B}} = (\tilde{B}_{-x}, \tilde{B}_{-y}) = \tilde{b}Q_{\tilde{A}} = (\tilde{b}\tilde{a})G$. So, if nothing has gone wrong in the previous steps, we should have $\tilde{A}_{-x}(\gamma) \equiv \tilde{B}_{-x}(\gamma) \pmod{p}$. But remember that PMNS is redundant, so there is no reason for the polynomials $\tilde{A}_{-x}$ and $\tilde{B}_{-x}$ to be the same. Since we want to stay in the PMNS, both use Algorithm 11 to compute the unique representation in $\mathcal{H}$. That is, Alice applies this algorithm to $\tilde{A}_{-x}$ and Bob applies it to $\tilde{B}_{-x}$. Since both polynomials represent the same element, they will compute exactly the same polynomial S in $\mathcal{H}$. This polynomial (or equivalently its coefficients) is the shared secret.

9.4 Application to the BrainpoolP256r1 curve

Now let us apply the above steps to the BrainpoolP256r1 curve. The parameters of the PMNS generated for the modulus of this curve are the following.

- $p = 76884956397045344220809746629001649093037950200943055203735601445031516197751$
- $n = 5$
- $\gamma = 36181054844448274924615137064228865317857967132027946769785381863627930216959$
- $\rho = 5057445902625897$
- $E(X) = X^5 - X + 1$
- $\phi = 2^{64}$
- $\delta = 10$

We have $\mathcal{G}$ a basis of $\mathcal{L}_{\mathcal{B}}$, with:

$$\mathcal{G} = \begin{pmatrix} 346810025099230 & 1808715194247007 & 865997808902389 & 49978327768209 & -1124833306030857 \\ 2155525219346237 & 865997808902389 & 49978327768209 & -1124833306030857 & -346810025099230 \\ -49978327768209 & 1174811633799066 & -778023280931627 & 1808715194247007 & 865997808902389 \\ 1124833306030857 & -778023280931627 & 1808715194247007 & 865997808902389 & 49978327768209 \\ -865997808902389 & 816019481134180 & 1174811633799066 & -778023280931627 & 1808715194247007 \end{pmatrix}$$

Note that the basis $\mathcal{G}$ does not need to be precomputed. The owner of the public parameters only has to specify the lattice basis reduction algorithm he used. The other

party can then use the same algorithm to generate the same reduced basis $\mathcal{G}$ on his side.

In the PMNS, the curve equation parameters a and b are as follows.

- $A(X) = 394545466801775X^4 + 91958636398934X^3 - 188654494615762X^2 + 568329300394517X - 64172238276606$
- $B(X) = 332273189684656X^4 - 27837284697370X^3 + 457751387181128X^2 + 137917419604169X + 403993650627198$

The affine coordinates of the base point G are the following.

- $G_x(X) = 709284413434711X^4 - 282565506868912X^3 - 414664387303631X^2 + 333190155266238X + 515682748159374$
- $G_y(X) = 752186478404743X^4 - 1303931212678822X^3 + 1031175002717100X^2 - 544298167880447X - 586070703183602$

The order of G is:

$$q = 76884956397045344220809746629001649092737531784414529538755519063063536359079$$

These elements are the ones to publish in the Public Key Certificates, along with the order q of the base point G .

Let us now show an execution of the ECDH protocol between Alice and Bob, where Alice uses Bob's public parameters.

First, Alice makes sure that the point G belongs to the BrainpoolP256r1 curve, as explained above.

Then, she randomly chooses $\tilde{a}$, with

$$\tilde{a} = 34084142494559515421294200630009964644620097218620410646808952075316095382448.$$

She performs the ECSM $Q_{\tilde{A}} = \tilde{a}G$ (with conversion in affine coordinates, in $\mathcal{H}$), where

- $Q_{\tilde{A}_x}(X) = -1007750800982065X^4 - 2952375322596485X^3 + 1743758398113388X^2 + 1006490078157683X - 1654746957699838$
- $Q_{\tilde{A}_y}(X) = -383477300708216X^4 - 1863076275775534X^3 + 2036234557905800X^2 + 502956056760792X - 1028830277236382$

She sends $Q_{\tilde{A}}$ to Bob.

Bob does the same. He randomly chooses $\tilde{b}$, with

$$\tilde{b} = 37474094135814049743963088622592531624656993137923336019815198752520708938766.$$

He performs the ECSM $Q_{\tilde{B}} = \tilde{b}G$, where

- $Q_{\tilde{B}_x}(X) = -944630697405954X^4 - 1203266558637530X^3 + 2634285015346200X^2 - 980520532268010X - 975821407818562$
- $Q_{\tilde{B}_y}(X) = -501668418993841X^4 - 1778793605110436X^3 + 2209645130510914X^2 + 874635364498249X - 1064046752736089$

He sends $Q_{\tilde{B}}$ to Alice.

Now, Alice has her secret $\tilde{a}$ and the point $Q_{\tilde{B}}$, while Bob has his secret $\tilde{b}$ and the point $Q_{\tilde{A}}$. Alice makes sure that $Q_{\tilde{B}} \in E(\mathbb{F}_p)$, while Bob makes sure that $Q_{\tilde{A}} \in E(\mathbb{F}_p)$, as explained above. Then, Alice computes $S_{\tilde{A}} = \tilde{a}Q_{\tilde{B}}$ and then computes the x affine coordinate $\tilde{A}_x$ as explained in Remark 14. We have:

$$\tilde{A}_x(X) = -394234700104963X^4 + 3218721931141113X^3 + 147447361220869X^2 - 2922656309100480X + 3081935956262716.$$

Bob computes $S_{\tilde{B}} = \tilde{b}Q_{\tilde{A}}$ and then the affine coordinate $\tilde{B}_x$. We have:

$$\tilde{B}_x(X) = 679086681474221X^4 + 6998606005990860X^3 - 2880743592766729X^2 - 2462841002293676X + 4669259653321784.$$

One can check that $\tilde{A}_x(\gamma) \equiv \tilde{B}_x(\gamma) \pmod{p}$. But, the polynomials are different. As the final step, Alice converts $\tilde{A}_x$ to the fundamental domain $\mathcal{H}$ and Bob on his side converts $\tilde{B}_x$ to $\mathcal{H}$, using Algorithm 11. They have:

- $\mathcal{H}(\tilde{A}_x)(X) = 133019607003417X^4 - 2819631945653144X^3 + 1561810634892407X^2 + 61146606402552X - 658192143701543$
- $\mathcal{H}(\tilde{B}_x)(X) = 133019607003417X^4 - 2819631945653144X^3 + 1561810634892407X^2 + 61146606402552X - 658192143701543$

So, they have the same polynomial which is the shared common secret.

9.5 C implementation and comparison with OpenSSL

In addition to our implementation using the SageMath library, we have implemented the aforementioned ECDH protocol in the C programming language. We compared it with two implementations that use the OpenSSL library. We made this choice because OpenSSL is an efficient C library that incorporates a variety of Brainpool curves. The first implementation is the aforementioned ECDH protocol that uses the OpenSSL BIGNUM structure, with the fast Montgomery modular operation implementations. This is relevant since our PMNS uses a Montgomery-like internal reduction method. The second implementation uses OpenSSL EC_POINT methods with precomputed parameters and data for each curve. This approach is the fastest (most efficient) and most high-level way to implement the ECDH protocol using this library. These implementations are available on GitHub at the following link.

<https://github.com/arithPMNS/ECDH-using-PMNS-and-OpenSSL>

Table 2 compares ECDH implementation using PMNS with its implementations using OpenSSL, for the BrainpoolP256r1, BrainpoolP320r1, and BrainpoolP384r1 curves.

Remark 17 (The choice of Brainpool curves). In elliptic curve cryptography, curves are usually built on very sparse primes (e.g., pseudo-Mersenne primes) for very efficient modular arithmetic. However, for security reasons, curves based on primes of no particular shape have been proposed, curves such as Brainpools. The work presented in this paper, particularly the **GMont-like** method, applies to all primes, regardless of their shape. This is why we chose Brainpool curves for our comparisons.

Regarding special primes, a recent work [DEMM⁺25] presents various methods to generate PMNS that are as efficient as pseudo-Mersenne primes. The paper includes implementations and comparisons. Applications to elliptic curves are also presented in that paper.

Before presenting the results, we first describe our measurement procedure for C language implementations. Measurements were performed on a HP EliteBook laptop, with:

- Processor: Intel 11th Gen Intel Core i5-1135G7@2.40GHz × 8
- Memory: 16 GiB of RAM
- OS: Ubuntu 20.04.6 LTS (64 bits)

The compiler is gcc version 9.4.0, the compiler options are as follows: `-Wall -O3 -lgmp -lcrypto`.

The test procedure is as follows:

- the *Turbo-Boost* is deactivated during the tests;

- 301 runs are executed in order to "heat" the cache memory, i.e. we ensure that the cache memory (data and instruction) is in an enough stabilized state in order to avoid untimely cache faults;
- one generates 51 random data sets, and for each data set the minimum of the execution clock cycle numbers over a batch of 301 runs is recorded;
- the performance is the average of all these minimums;
- this procedure is run on console mode, to avoid system perturbations, and obtain the most accurate cycle counts.

The clock cycle number is obtained using the `rdtsc` instruction which loads the current value of the processor's time-stamp counter into a 64-bit register. The processor monotonically increments the time-stamp counter every clock cycle. Hence calling this instruction before and just after a sequence of instructions allows to obtain the number of elapsed cycles.

Table 2: Clock cycle number comparisons of ECDH for some Brainpool curves.

	BrainpoolP256r1	BrainpoolP320r1	BrainpoolP384r1
ECDH using PMNS	4 173 173	6 783 652	10 454 174
ECDH using OpenSSL BIGNUM	15 717 289	22 386 643	29 811 327
ECDH using OpenSSL EC_POINT	5 017 823	8 468 672	12 647 578

As can be seen in Table 2, the ECDH implementation using PMNS is much faster than the implementation using OpenSSL BIGNUM, and slightly faster than the implementation using OpenSSL EC_POINT. For OpenSSL BIGNUM, this can be explained by two factors. First, previous studies (see [DDV20, DDMG25]) have demonstrated that PMNS is faster than OpenSSL's Montgomery modular multiplication method. The same applies to simple addition and subtraction since there is no carry propagation in PMNS. Second, the elliptic curve point doubling and addition formula involves multiple additions and subtractions. Using OpenSSL BIGNUM, some of these additions and subtractions must be followed by a modular reduction to obtain the correct results for points addition and doubling. This is not the case when using the PMNS we generated, because we already used one more coefficient than OpenSSL to represent the elements of $\mathbb{Z}/p\mathbb{Z}$.

For OpenSSL EC_POINT, we used the corresponding named curves with all the pre-computation and optimisations done for the elliptic curve scalar multiplication and other additional optimisations. This explains why it is much faster than the ECDH implementation using OpenSSL BIGNUM. Still, it can be seen that PMNS remains slightly faster.

Remark 18 (Memory usage). As mentioned above, for each curve modulus, the PMNS that we generated uses one more coefficient (of 64 bits) than the OpenSSL BIGNUM to represent the elements of $\mathbb{Z}/p\mathbb{Z}$. However, the BIGNUM structure uses additional data and flags, which makes it use more memory than the PMNS to represent the elements (see [bn_internal](#)). Table 3 summarizes the amount of memory (in bits) required to represent the elements of $\mathbb{Z}/p\mathbb{Z}$ for each curve modulus. Note that OpenSSL EC_POINT also uses the BIGNUM structure.

10 Conclusion

In this paper, we have presented **GMont-like**, a generalised Montgomery-like internal reduction method. We discussed a number of properties of this method. With **GMont-like**,

Table 3: Number of bits used to represent the elements of $\mathbb{Z}/p\mathbb{Z}$ for some Brainpool curves.

	BrainpoolP256r1	BrainpoolP320r1	BrainpoolP384r1
PMNS	320	384	448
OpenSSL BIGNUM	384	448	512

we also improved the memory requirement to represent PMNS elements. In addition, we explained how to compute element representations in some special domains. This allowed us to make a precise study of redundancy in the PMNS, with a process to compute the desired representation of any element $a \in \mathbb{Z}/p\mathbb{Z}$. Based on some properties of **GMont-like**, we explained how to perform equality test within the PMNS. This for instance removes the need for conversion to the original system in order to apply cryptographic countermeasures that require an arithmetic equality test. We provided implementations to generate PMNS and to check the work presented in this paper. This work makes the PMNS a more complete system. Finally, as an application, we showed how to use the ECDH key agreement protocol with PMNS, without requiring another number system. We provided an implementation using the SageMath library. We also provided a C implementation and compared it with the OpenSSL library, for some Brainpool curves. Our codes are available at:

- <https://github.com/arithPMNS/ECDH-using-PMNS-and-OpenSSL>
- <https://github.com/arithPMNS/ECDH-with-PMNS>
- <https://github.com/arithPMNS/PMNS-and-redundancy>

This is just one application. It can be easily extended any curve and also to several other elliptic curve cryptographic protocols such as ECDSA or Pairing-based crypto-protocols. This work also makes it more interesting to consider using PMNS as an alternative to the classical representation in post-quantum schemes such as SQIsign and CRYSTALS-Dilithium. Finally, it remains to be seen how the redundancy of PMNS could protect the implementation when considering side-channel analysis. Masking is often used as a countermeasure, but it is expensive. PMNS could provide the same level of security at a lower cost. Efficient software or hardware implementations of the work presented in this paper remain to be studied, since our C implementation can be improved in many ways. For example, it could take advantage of the PMNS's high parallelisation capability.

References

- [AVL19] R. Abarzúa, C. Valencia, and J. López. Survey for performance and security problems of passive side-channel attacks countermeasures in ECC. Cryptology ePrint Archive, Report 2019/010, 2019.
- [Bar] Paul Barrett. Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor. 311– 323 (Springer, Berlin, Heidelberg, 1987).
- [BIPa] Jean-Claude Bajard, Laurent Imbert, and Thomas Plantard. Arithmetic operations in the polynomial modular number system. 206–213, 17th IEEE Symposium on Computer Arithmetic (ARITH-17 2005), 27-29.
- [BIPb] Jean-Claude Bajard, Laurent Imbert, and Thomas Plantard. Modular number systems: Beyond the Mersenne family. 159–169 (Springer Berlin Heidelberg, Berlin, Heidelberg, 2005).

- [BL] Daniel J. Bernstein and Tanja Lange. Explicit-formulas database: Jacobian coordinates for short weierstrass curves. <http://hyperelliptic.org/EFD/g1p/auto-shortw-jacobian.html>.
- [BMPV22] Jean-Claude Bajard, Jérémy Marrez, Thomas Plantard, and Pascal Véron. On polynomial modular number systems over $\mathbb{Z}/p\mathbb{Z}$. *Advances in Mathematics of Communications*, 0:–, 2022.
- [BOS] Johannes Blömer, Martin Otto, and Jean-Pierre Seifert. Sign change fault attacks on elliptic curve cryptosystems. Vol. 4236 of *Lecture Notes in Computer Science*, 36–52 (Springer, 2006).
- [CDD⁺21] Asma Chaouch, Laurent-Stéphane Didier, Fangan-Yssouf Dosso, Nadia El Mrabet, Belgacem Bouallegue, and Bouraoui Ouni. Two hardware implementations for modular multiplication in the AMNS: sequential and semi-parallel. *J. Inf. Secur. Appl.*, 58:102770, 2021.
- [Cra92] Richard E Crandall. Method and apparatus for public key exchange in a cryptographic system, October 27 1992. US Patent 5,159,632.
- [DDE⁺] Laurent-Stéphane Didier, Fangan-Yssouf Dosso, Nadia El Mrabet, Jérémy Marrez, and Pascal Véron. Randomization of arithmetic over polynomial modular number system. 26th IEEE Symposium on Computer Arithmetic, ARITH 2019, 199–206.
- [DDMG25] Fangan Yssouf Dosso, Sylvain Duquesne, Nadia El Mrabet, and Emma Gautier. PMNS arithmetic for elliptic curve cryptography. *Cryptology ePrint Archive*, Paper 2025/467, 2025.
- [DDV20] Laurent-Stéphane Didier, Fangan Yssouf Dosso, and Pascal Véron. Efficient modular operations using the Adapted Modular Number System. *Journal of Cryptographic Engineering*, pages 1–23, 2020.
- [DEMM⁺25] Fangan Yssouf Dosso, Nadia El Mrabet, Nicolas Méloni, François Palma, and Pascal Véron. Friendly primes for efficient modular arithmetic using the Polynomial Modular Number System. *Journal of cryptographic engineering*, 15(18), 2025.
- [DKL⁺18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR TCHES*, 2018(1):238–268, 2018.
- [Dos20] Fangan Yssouf Dosso. *Contribution de l'arithmétique des ordinateurs aux implémentations résistantes aux attaques par canaux auxiliaires*. Theses, Université de Toulon, April 2020.
- [DRDE] Laurent-Stéphane Didier, Jean-Marc Robert, Fangan-Yssouf Dosso, and Nadia El Mrabet. A software comparison of RNS and PMNS. 29th IEEE Symposium on Computer Arithmetic, ARITH 2022.
- [DRV22] Fangan Yssouf Dosso, Jean-Marc Robert, and Pascal Véron. PMNS for efficient arithmetic and small memory cost. *IEEE Transactions on Emerging Topics in Computing*, 10(3):1263–1277, 2022.
- [EJ17] Nadia El Mrabet and Marc Joye. *Guide to pairing-based cryptography*. CRC Press, 2017.

- [FS07] David E. Fu and Jerome Solinas. IKE and IKEv2 Authentication Using the Elliptic Curve Digital Signature Algorithm. RFC 4754, January 2007.
- [Ga] Torbjörn Granlund and al. GNU multiple precision arithmetic library 6.1.2. <https://gmplib.org/>.
- [Gar59] H. L. Garner. The residue number system. *IRE Transactions on Electronic Computers*, EL 8(6):140–147, 1959.
- [GJM⁺11] Raveen R Goundar, Marc Joye, Atsuko Miyaji, Matthieu Rivain, and Alexandre Venelli. Scalar multiplication on weierstraß elliptic curves from co-z arithmetic. *Journal of cryptographic engineering*, 1(2):161, 2011.
- [Joy19] Marc Joye. Protecting ECC against fault attacks: The ring extension method revisited. Cryptology ePrint Archive, Paper 2019/495, 2019.
- [JY02] Marc Joye and Sung-Ming Yen. The montgomery powering ladder. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 291–302. Springer, 2002.
- [KI77] Aleksandr Korkine and Zolotareff Yegor Ivanovich. Sur les formes quadratiques positives. *Mathematische Annalen*, 11(2):242–292, 1877.
- [Kob87] Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of computation*, 48(177), 1987.
- [LL97] Chae Hoon Lim and Pil Joong Lee. A key recovery attack on discrete log-based schemes using a prime order subgroup. In Burton S. Kaliski, editor, *Advances in Cryptology — CRYPTO ’97*, pages 249–263, Berlin, Heidelberg, 1997. Springer Berlin Heidelberg.
- [LLL82] Arjen K Lenstra, Hendrik Willem Lenstra, and László Lovász. Factoring polynomials with rational coefficients. *Mathematische annalen*, 261:515–534, 1982.
- [Mél22] Nicolas Méloni. An Alternative Approach to Polynomial Modular Number System Internal Reduction. *IEEE Transactions on Emerging Topics in Computing*, July 2022.
- [Mil85] Victor S Miller. Use of elliptic curves in cryptography, 1985. 417–426 (Springer, 1985).
- [Mon85] Peter L. Montgomery. Modular multiplication without trial division. *Mathematics of Computation*, 44(170):519–521, 1985.
- [NEPV24] Louis Noyez, Nadia El Mrabet, Olivier Potin, and Pascal Véron. Modular multiplication in the AMNS representation: Hardware implementation. In Maria Eichlseder and Sébastien Gambs, editors, *Selected Areas in Cryptography - SAC 2024 - 31st International Conference, Montreal, QC, Canada, August 28-30, 2024, Revised Selected Papers, Part II*, volume 15517 of *Lecture Notes in Computer Science*, pages 269–293. Springer, 2024.
- [NP08] Christophe Negre and Thomas Plantard. Efficient modular arithmetic in adapted modular number system using lagrange representation, 2008. 463–477 (Springer Berlin Heidelberg, 2008).
- [NSS04] David Naccache, Nigel P Smart, and Jacques Stern. Projective coordinates leak. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 257–267. Springer, 2004.

-
- [Pro] The OpenSSL Project. Openssl. <https://www.openssl.org/>.
- [RSA78] Ronald L Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.
- [Sa] William Stein and al. Sagemath. <http://www.sagemath.org/index.html>.
- [SE91] Claus-Peter Schnorr and Martin Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. In *International Symposium on Fundamentals of Computation Theory*, pages 68–85. Springer, 1991.
- [Sol99] Jerome A Solinas. *Generalized Mersenne numbers*. Citeseer, 1999.